

Cumhuriyet Üniversitesi

İşletim Sistemleri Dersi

Bölüm 7

Ana Bellek

(Main Memory)

Dr. Halil ARSLAN

Bölümün hedefleri

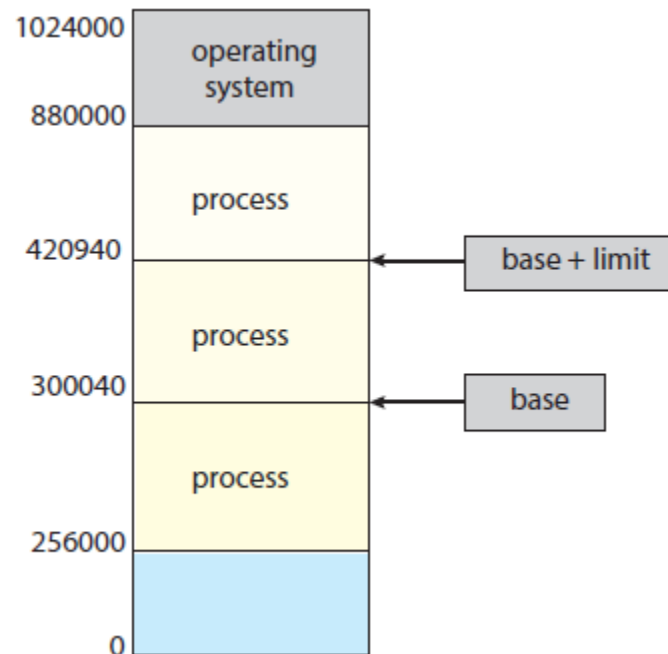
- Ana bellek konsepti
- Bitişik bellek tahsisi (Contiguous memory allocation)
- Sayfalı bellek tahsisi - Sayfalama (Paging)
- Sayfa tablosu yapıları (Page table)
- Takaslama (Swapping)
- Intel 32 ve x86-64 Mimarileri
- ARMv8 Mimarisi

Bellek Yönetimi - (Memory Management)

- Programların çalıştırılması için (diskten) belleğe getirilmesi (en azından bir kısmı) ve bir sürece (process) dönüştürülmesi gerekir.
- Ana bellek ve kaydediciler CPU'nun doğrudan erişebildiği depolama birimleridir.
- Bellek birimi yalnızca şu akışları görür:
 - adresler + okuma istekleri
 - adres + veri + yazma istekleri
- Register erişimi, CPU saat hızında (veya daha az) gerçekleşir
- Ana belleğe erişim daha yavaş olabilir ve bu bir gecikmeye neden olabilir (**stall**)
- Önbellekler (**cache**), ana bellek ve CPU kaydedicileri arasında yer alır.
- Uygun çalışmayı sağlamak için belleğin korunması gerekir.

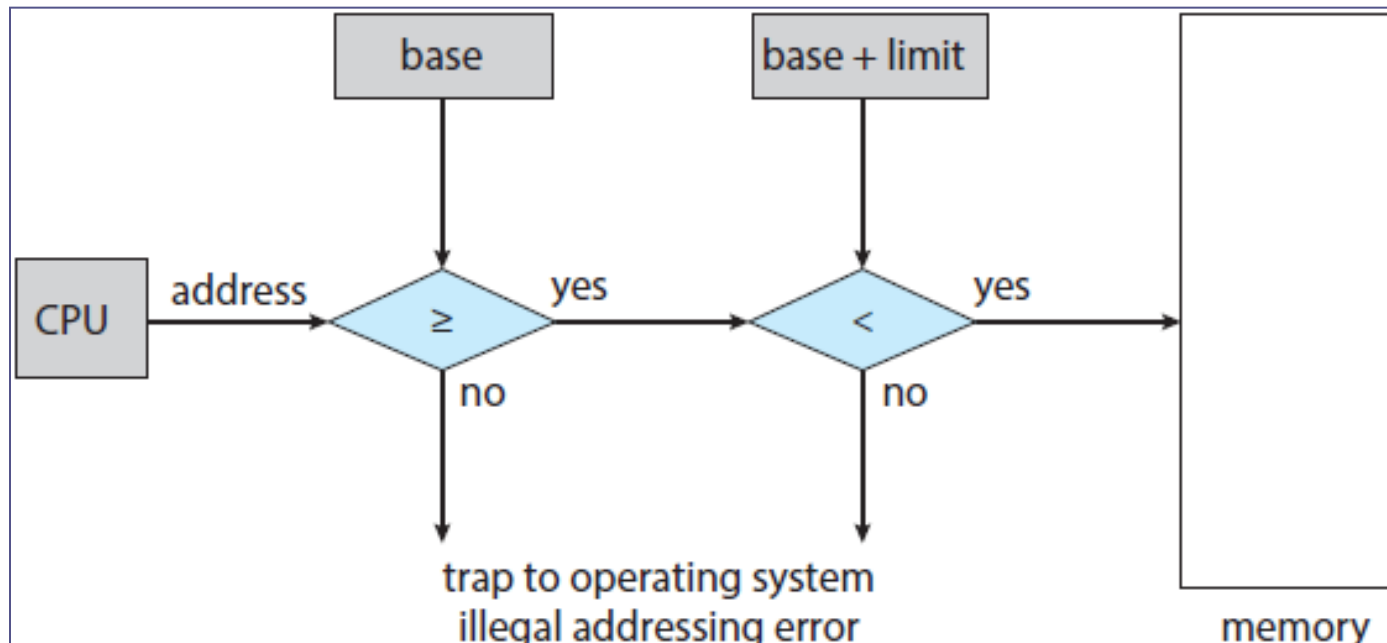
Koruma - Protection

- Bir sürecin sadece, adres uzayındaki adreslere erişebilmesi gerekir.
 - Diğer süreçlerin bellek alanları korunmalıdır.
- Koruma, bir sürecin mantıksal adres uzayı tanımlanarak sağlanabilir.
 - Bu tanımlama bir çift **Base** ve **Limit** register'ları kullanılarak sağlanır.



Donanım adreslerinin korunması

- CPU, kullanıcı modunda oluşturulan her bellek erişiminin **base** ve **limit** arasında olduğunu kontrol eder.
- **Base** ve **limit register**'lerini yükleyen komutlar ayrıcalıklıdır ve bu register'lar işletim sistemi tarafından (kernel mode) yüklenir.



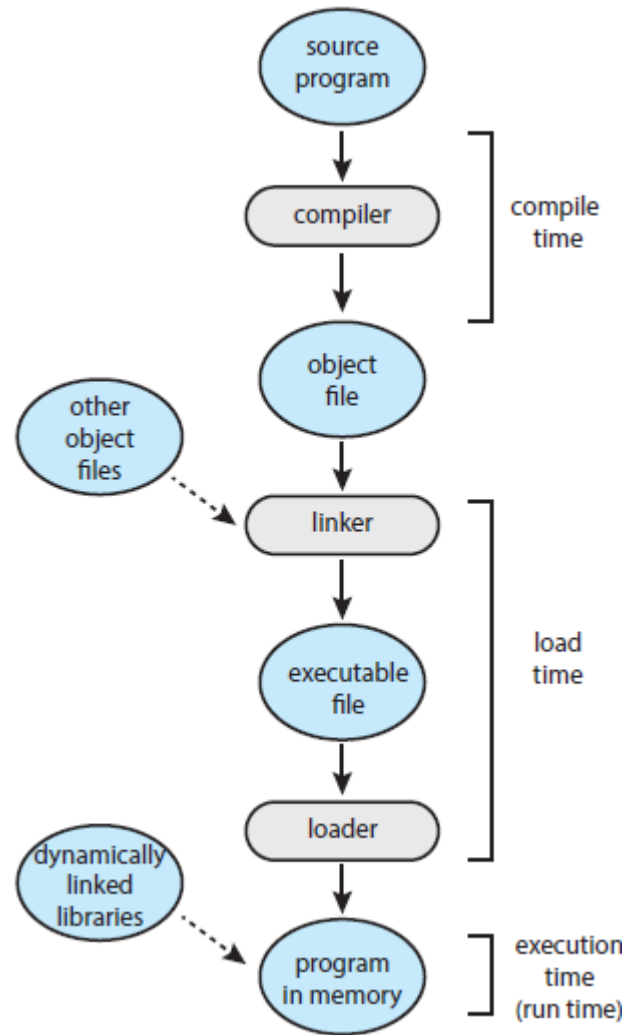
Adres Atama (Address Binding)

- Programlar diskte çalıştırılabilir dosya (binary executable file) olarak tutulur.
- Diskteki programlar, girdi kuyruğundan (**input queue**) çalıştırılmak üzere belleğe yüklenirler.
 - Çoğu sistem, kullanıcı süreçlerinin fiziksel belleğin herhangi bir bölümünde bulunmasına izin verir.
 - Bilgisayarın adres alanı 00000'dan başlasa da, kullanıcı süreçlerinin ilk adresinin 00000 olması gerekmez.
- Bir program farklı aşamalarda farklı adreslerle temsil edilir,
 - Kaynak kod adresleri genellikle semboliktir
 - Derleme sürecinde yeniden atanabilir adreslere bağlanır
 - Bağlayıcı veya yükleyici (linker/loader), yeniden atanabilir adresleri mutlak adreslere dönüştürür
 - Her adres ataması, bir adres uzayını diğerine haritalar.

Adres Atama (Address Binding)

- Komutların ve verilerin bellek adreslerine bağlanması üç farklı aşamada gerçekleşebilir;
 - **Derleme zamanı (Compile):** Bellek konumu önceden biliniyorsa, mutlak kod (**absolute code**) üretilebilir; başlangıç konumu değişirse kod yeniden derlenmelidir.
 - **Yükleme zamanı (Load):** Derleme zamanında bellek konumu bilinmiyorsa yeniden konumlandırılabilir kod (**relocatable code**) üretilir.
 - **Yürütme zamanı (Execution):** İşlem yürütülürken bir bellek bölümünden diğerine taşınabiliyorsa adres atama çalışma zamanına kadar ertelenir. İşletim sistemleri bu yöntemi kullanır.
 - Adresleme için donanım desteğine ihtiyaç vardır (ör. Base ve Limit register'ları)

Adres Atama (Address Binding)

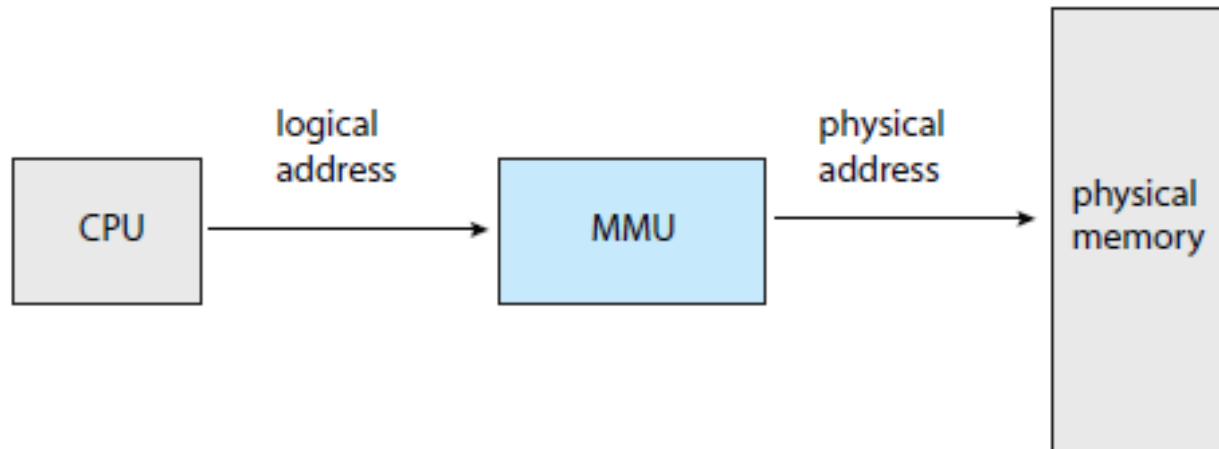


Fiziksel ve Mantıksal Adresler

- Ayrılmış her bir fiziksel adres uzayına bağlı mantıksal adres uzayı kavramı, **bellek yönetimi konseptinin temelidir**.
 - Mantıksal adres (**Logical address**): CPU tarafından oluşturulan adreslerdir.
 - Fiziksel adres (**Physical address**): Bellek birimi tarafından kullanılan adres
- Mantıksal (logical) ve fiziksel adresler, derleme (**compile-time**) ve yükleme zamanı (**load-time**)’nda (adres atama şeması) aynıdır.
- Mantıksal (logical) ve fiziksel adresler, yürütme zamanında (**execution-time**) (adres atama şeması) farklıdır.
- Mantıksal adres uzayı (**logical address space**), bir program tarafından oluşturulan tüm mantıksal adreslerin kümesidir.
- Fiziksel adres uzayı (**physical address space**), bir program tarafından üretilen tüm fiziksel adreslerin kümesidir.

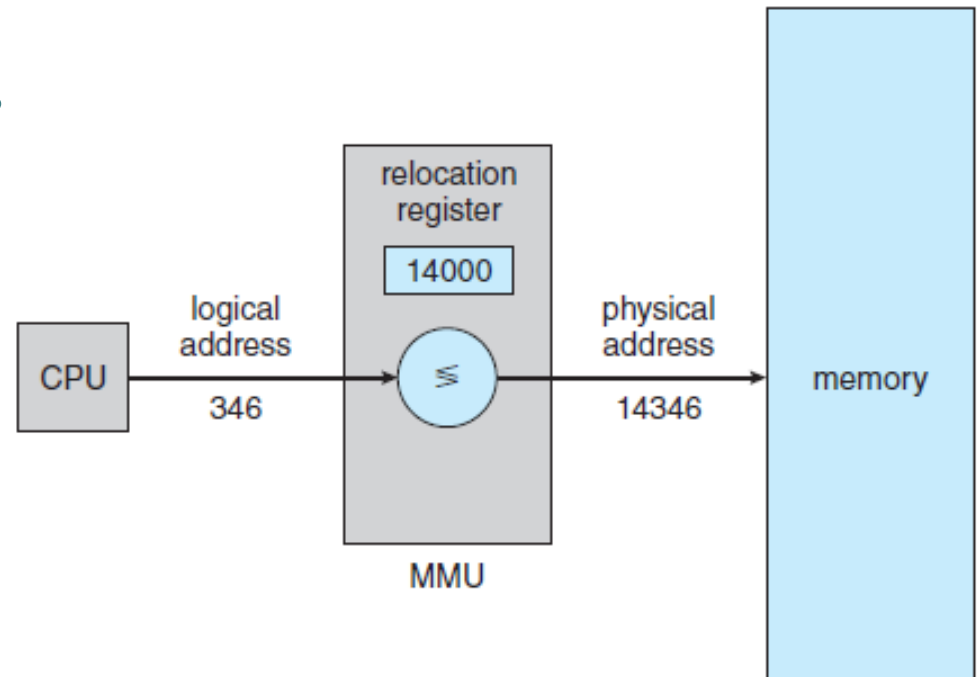
Memory-Management Unit (MMU)

- Çalışma zamanında (execution-time) mantıksal adreslerle fiziksel adresleri eşleyen donanımdır. (logical → physical)



Memory-Management Unit (MMU)

- Base register yeniden konumlandırma register'ı (**relocation register**) olarak adlandırılır.
- Bir kullanıcı süreci tarafından üretilen her adrese, relocation register'daki değer eklenecektir.
- Kullanıcı programı mantıksal (**logical**) adreslerle ilgilenir; Asla gerçek fiziksel (**real physical**) adresleri bilmez
 - Yürütme zamanı adres ataması, bellekteki konuma referans yapıldığında gerçekleşir.
 - Fiziksel adreslere bağlı mantıksal adresler şeklinde tasarlanır



Dynamic Loading

- Programların çalıştırılabilmesi için hafızada olmaları gerekir
- Program bileşenleri (rutin) çağrılınca kadar belleğe yüklenmez.
 - Daha iyi bellek alanı kullanımı için kullanılmayan rutinler yüklenmemeli.
- Tüm rutinler diskte yeniden yerleştirilebilir yükleme formunda (relocatable load format) tutulur.
- Bu yaklaşım, seyrek meydana gelen durumları yönetmek için büyük miktarda kod gerektiğinde kullanışlıdır.
- İşletim sistemine özel bir durum değildir
 - Program tasarımı sürecinde gerçekleştirilir
 - İşletim sistemi, dinamik yüklemeyi uygulamak için kütüphaneler sunarak bu sürece yardımcı olabilir.

Dynamic Linking

- **Statik bağlama (Static linking):** Yükleyici (loader) tarafından sistem kütüphaneleri ve program kodlarının ikili program imajlarıyla birleştirilmesini ifade eder.
- **Dinamik bağlama (Dynamic linking):** Linking sürecinin programın çalışma (execution-time) zamanında gerçekleştirilmesini ifade eder.
- Dynamically Linked Libraries (**DLLs**)'ler, program çalıştırıldığında kullanıcı programlarına bağlanan sistem kütüphaneleridir.
 - Bir program dinamik kütüphanede bulunan bir rutini çağırdığında yükleyici DLL'i bulur ve belleğe yükler
 - Ardından bu rutinleri çağıran adresleri DLL'in yüklendiği bellek uzayı ile ayarlar
 - Dynamic linking Windows ve Linux tarafında yaygın olarak kullanılır
- Dinamik linking, özellikle yazılım kütüphaneleri için kullanışlıdır
 - Paylaşılan kütüphaneler (**shared libraries**) olarak da bilinir.
 - Bu kütüphaneler güncellenerek bakım/dağıtım kolaylaşır.

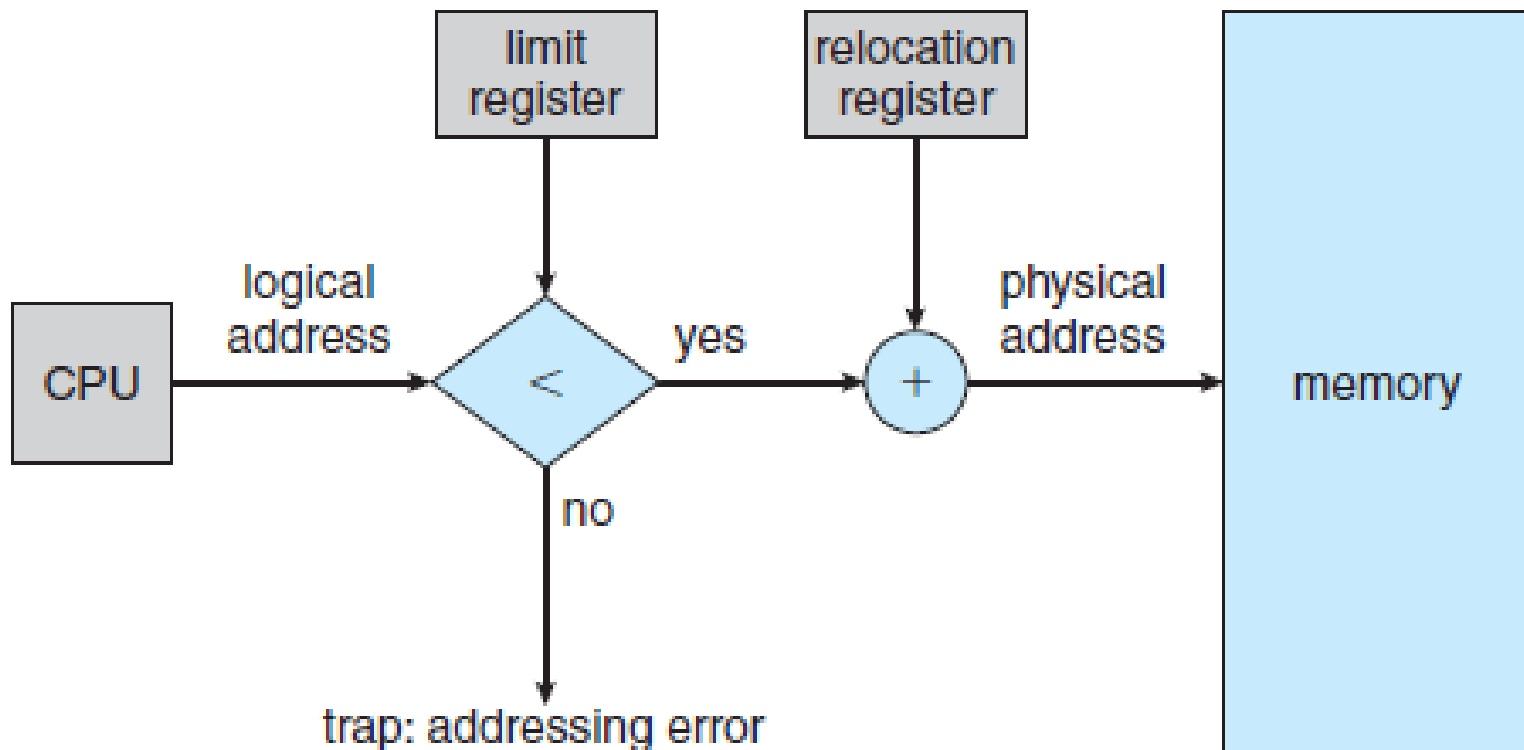
Bitişik Bellek Tahsisi- Contiguous Memory Allocation

- Ana bellek hem işletim sistemini hem de kullanıcı süreçlerini barındırmalıdır.
- Ana bellek sınırlı bir kaynak olduğu için verimli bir şekilde tahsis edilmelidir
- Ana bellek genellikle iki bölüme ayrılır:
 - İşletim sistemi
 - Kullanıcı süreçleri
- İşletim sistemi düşük yada yüksek bellek adreslerine yerleştirilebilir.
 - Bu, kesme vektörlerinin adres konumları gibi bir çok faktöre bağlıdır.
 - Ancak Linux ve Windows yüksek bellek adreslerine yerleştirilir.
- Bitişik bellek tahsisi ilkel bir yöntemdir.
- Bitişik bellek tahsisinde, her süreç, bir sonraki süreci içeren bölüme bitişik olan tek bir bellek bölümünde yer alır.

Bitişik Bellek Tahsisi – Memory Protection

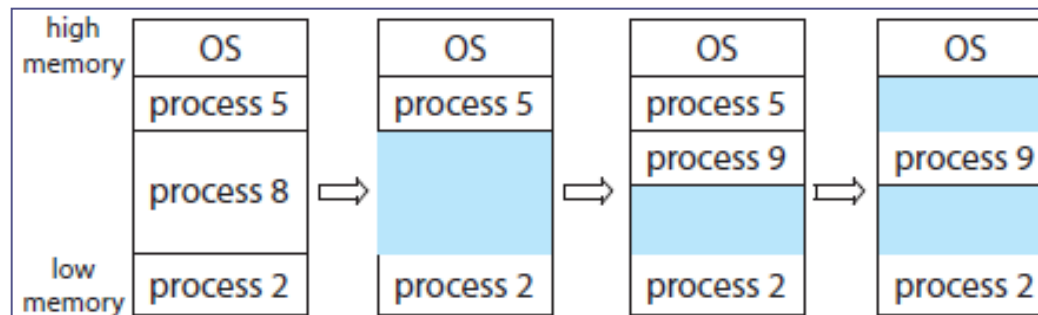
- Bir süreç sahip olmadığı bir bellek alanına erişmek istediğinde bu alan **korunmalıdır**.
- Bitişik bellek tahsisinde bu işlem **Base (relocation register)** ve **Limit register**'ları ile sağlanır.
 - Relocation register (base), en küçük fiziksel bellek adresini içerir.
 - Limit register'ı, mantıksal adres aralığını içerir.
 - Bu durumda her mantıksal adres, limit register'ı tarafından belirlenen adres aralığında olmalıdır
- CPU planlayıcısı yürütmek için bir süreç seçtiğinde context swiching bu sürecin base ve limit register'larını doğru değerlerle yükler.
- Erişilmek istenen adresler bu register'lardaki değerlerle kontrol edildiği için hem işletim sistemi hem de diğer programların bellek uzayları korunmuş olur.

Bitişik Bellek Tahsisi – Memory Protection



Bitişik Bellek Tahsisi – Memory Allocation

- Bellek tahsisinin en basit yolu, her bölümün tek bir süreç içerebileceği değişken boyutlu bölümlere atanmasıdır. (**variable partition scheme**)
 - Başlangıçta tüm bellek boştur ve tek bir boşluk (hole) olarak kabul edilir.
 - İlerleyen zamanlarda bu boşluklar parçalı hale gelir.
 - İşletim sistemleri tahsis edilen ve boş bellek alanlarını **allocated partitions** ve **free partitions (hole)** olarak tutar.
 - Bir süreç yüklenmek istendiğinde, işletim sistemi bu süreç için gereken boyutta boşluk (**hole**) arar. Varsa yerleştirir yoksa bekletebilir (reddedebilir de).
 - Bir süreç sonlandığında bellek serbest bırakılır ve hole'ler bitişikse birleşebilir ve daha büyük hole'ler elde edilir.



Bitişik Bellek Tahsisi – Memory Allocation

- Bir program çalıştırıldığında bu sürece hangi bellek bloğunun tahsis edileceğini belirlemek gerekir (**dynamic storage-allocation problem**)
 - Serbest partisyonlardan n uzunluklu alanın nasıl karşılanacağı belirlenmeli!
 - Bu sorunun pekçok çözümü vardır;
 - **First fit (ilk uygun):** Boyutu uygun ilk alan tahsis edilir. Sıralama gereksinimi yok. İlk bulunan alandan sonra arama sonlandırılır.
 - **Best fit (en uygun):** Yeterince büyük olan en küçük alan tahsis edilir. Kalan alan çok küçük olacaktır. Liste sıralı olmalı yoksa tüm liste taranmalı.
 - **Worst fit (en kötü uygun):** En büyük boşluktan bellek tahsisi yapılır. Kalan alan Best Fit'e göre daha yararlı olabilir. Liste sıralı olmalı yoksa tüm liste taranmalı.
 - Genellikle **First Fit** daha hızlıdır. Depolama uyumu açısından simülasyonlarda First Fit ve Best Fit daha iyi sonuçlar vermiştir.

Bitişik Bellek Tahsisi – Fragmentation

- Süreçler belleğe yüklenirken ve bellekten kaldırılırken boş bellek alanları küçük parçalara bölünür.
- **Dışsal parçalanma (external fragmentation)**, yeterli toplam bellek alanı olmasına rağmen mevcut boş alanlar bitişik olmadığı için talebin karşılanamamasıdır.
 - Hem first-fit hem de best-fit stratejileri **dışsal parçalanmaya** neden olurlar.
- Dışsal parçalanmayı önlemeye yönelik olarak geliştirilen yaklaşımlardan birisi fiziksel belleğin sabit boyutlu bloklara ayrılarak süreçlere tahsis edilmesidir.
 - Fiziksel bellek birimler halinde tahsis edilse de bir işleme ayrılan bellek minimum blok boyutundan dolayı istenen bellekten büyük olabilir.
 - Bu durum **İçsel Parçalanma (internal fragmentation)**, olarak ifade edilir.
- Dışsal parçalanmayı önlemenin bir yolu da bütünleştirmedir (**compaction**). Amaç küçük boşlukları bir araya getirmektedir.
 - Bütünleştirme her zaman mümkün olmaz (**Static Relocation**). Ayrıca maliyetlidir.
- Başka bir çözüm ise **paging**'tir.

Sayfalı Bellek Tahsisi - Paging

- Şimdiye kadar anlatılan bellek yönetimi, bir sürecin fiziksel adres uzayının bitişik (tek parça) olmasını gerektirmektedir.
- **Paging** yöntemi ise fiziksel adres uzayının bitişik olmasının zorunlu olmadığı bir bellek yönetim modelidir.
- Paging modeli, dışsal parçalanmayı ve buna bağlı olarak bütünleştirmeyi gerektirmeyen bir yöntemdir.
 - Bu ve benzeri pekçok avantajından dolayı paging yöntemi çoğu işletim sistemi ve platform tarafından tercih edilir.
- Paging tekniği, işletim sistemi ve donanım arasında sıkı bir işbirliği ile gerçekleştirilir.
- Paging'in temeli, fiziksel belleği **frame (çerçeve)** adı verilen sabit boyutlu bloklara ayırarak mantıksal belleği aynı boyutlu **pages** adı verilen bloklarla eşlemeyi içerir.
- Bir süreç yürütüldüğünde, bu sürecin sayfaları kullanılabilir fiziksel bellek çerçevesine yüklenir. (dosya sisteminden yada yardımcı belleklerden)

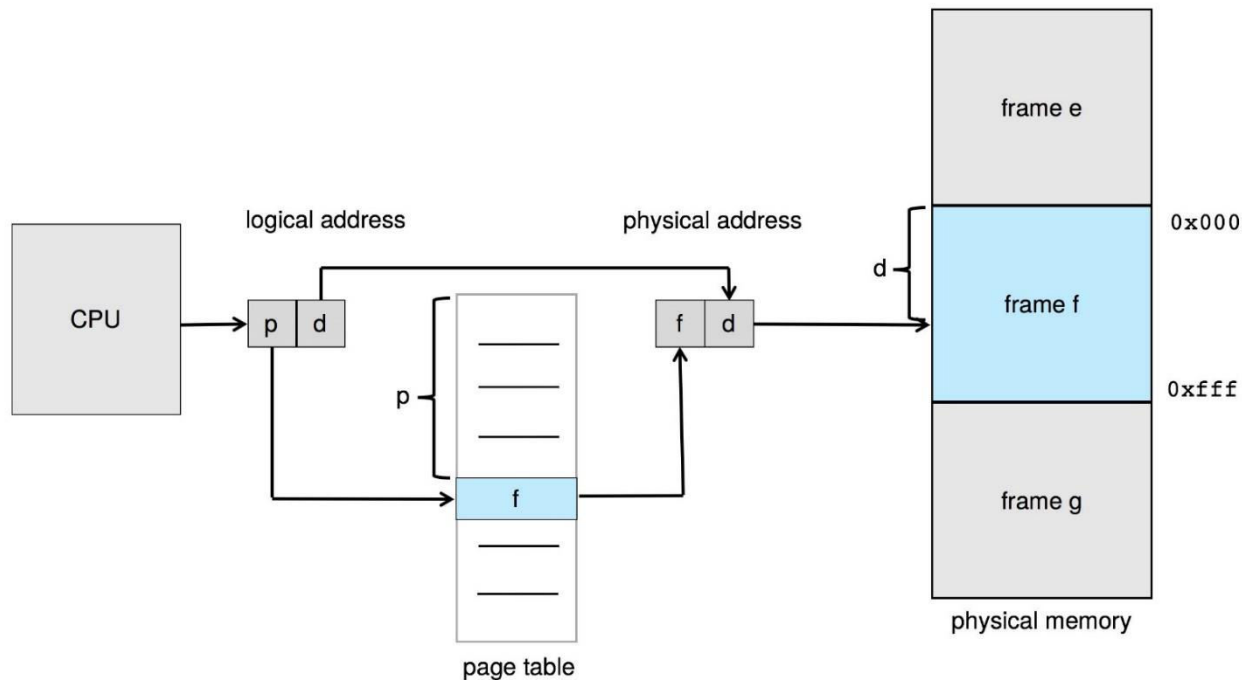
Sayfalı Bellek Tahsisi - Paging

- Yardımcı bellek (backing store), bellek çerçeveleriyle aynı boyutta olan sabit boyutlu bloklara bölünmüştür.
- Bu fikir, mantıksal adres uzayı ile fiziksel bellek uzayını tamamen ayırmıştır.
 - Fiziksel bellek kapasitesi çok az olsa da 64 bitlik bir mantıksal adres uzayı olabilir.
- CPU tarafından üretilen her adres iki bölüme ayrılmıştır.
 - (a) Sayfa numarası (page number-p), (b) Sayfa konumu (page offset-d)



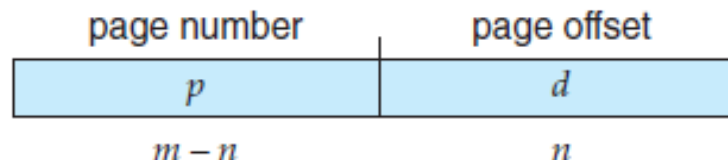
Sayfalı Bellek Tahsisi - Paging

- **Page table:** Fiziksel bellekteki her çerçevenin base adresini içerir.
- **Page number (p):** Sayfa numarası, page table'daki sayfanın sıra numarası.
- **Page offset (d):** Sayfa konumu, bellek birimine gönderilen sayfa numarasından fiziksel bellek adresini tanımlamak için base adresle birleştirilir.
- Fiziksel adres için çerçevenin base adresi sayfa offseti ile birleştirilir.



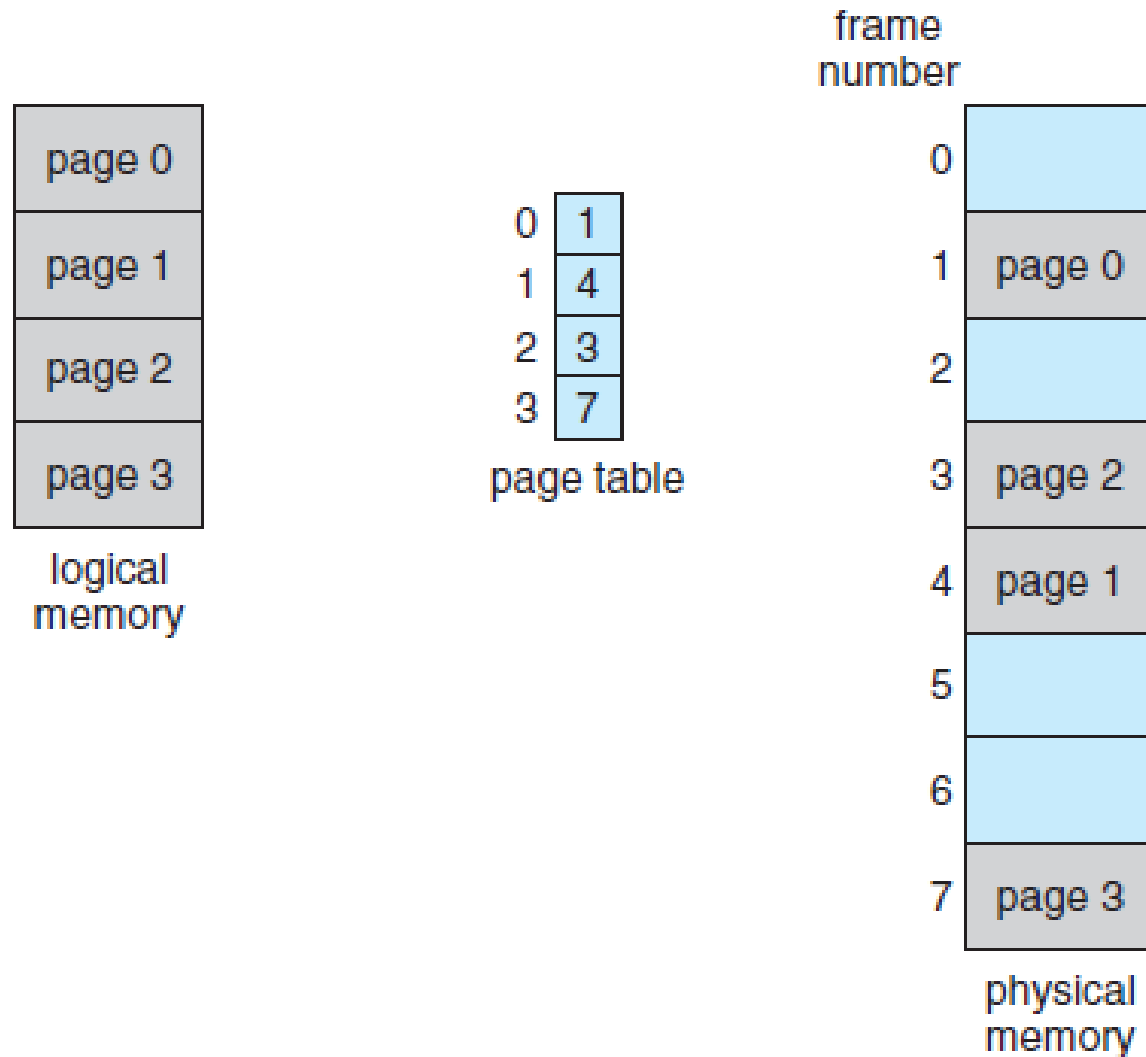
Sayfalı Bellek Tahsisi - Paging

- **Page size** (sayfa boyutu yada çerçeve boyutu), donanım tarafından belirlenir.
- Bir sayfanın boyutu, bilgisayar mimarisine bağlı olarak 4 KB ile 1 GB arasında değişebilir.
- Sayfa boyutu olarak 2'nin kuvvetinin seçilmesi, mantıksal adresin sayfa numarasına ve ofsetine çevrilmesini kolaylaştırır.
- Mantıksal adres uzayının boyutu 2^m ise ve bir **sayfa boyutu** 2^n bayt ise;
 - Mantıksal adresin yüksek değerlikli $m-n$ bitleri sayfa numarasını
 - Düşük değerlikli n bitleri sayfa ofsetini belirler.
- Dolayısıyla mantıksal adres aşağıdaki gibidir:



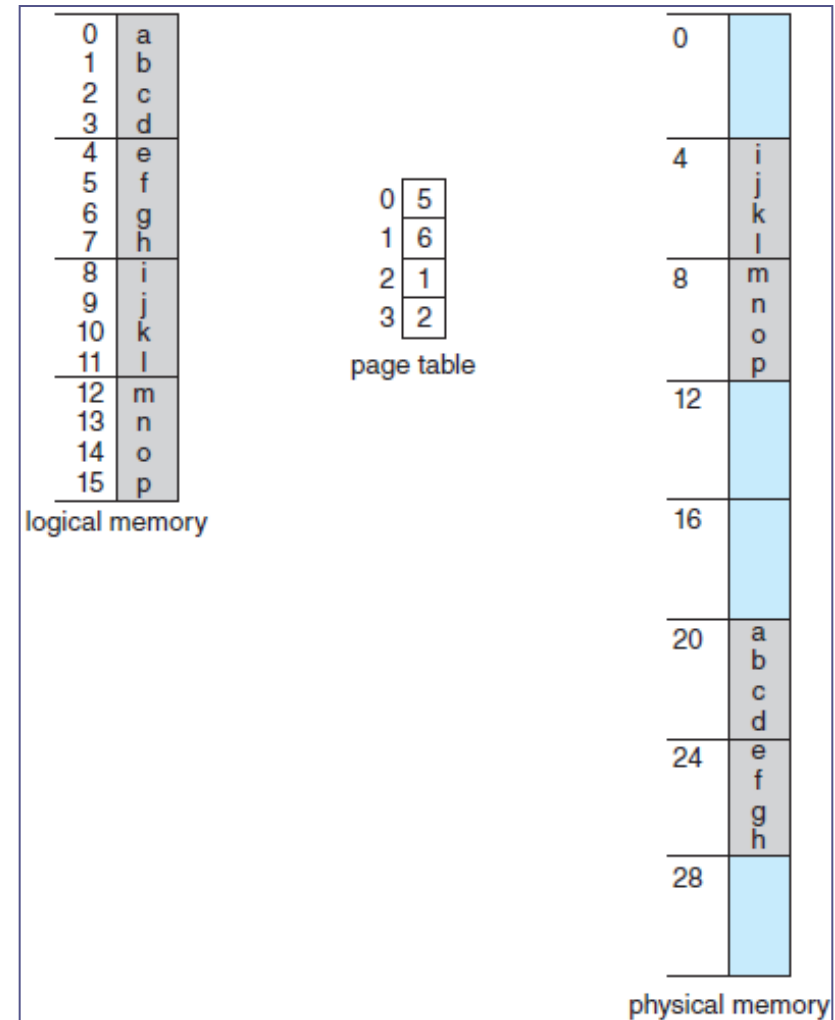
- Buradaki **p**, page table'daki bir indistir ve **d**, sayfa içindeki uzaklıktır.

Sayfalı Bellek Tahsisi - Paging



Sayfalı Bellek Tahsisi - Paging

- $n=2$ ve $m=4$ 'lük bir mantıksal belleğin, 32 byte'lık bir fiziksel belleğe eşlemesi;
 - Mantıksal adres 0 için (a değeri):
 - $\text{page}=0$, $\text{offset}=0$ 'dır.
 - $\text{page_table}[0]$ 'ın değeri 5, $\text{page size}=4$ ise
 - $(\text{p_num} \times \text{p_size}) + \text{offset} \rightarrow 5 \times 4 + 0 = \mathbf{20}$
 - Mantıksal adres 3 için (d değeri):
 - $\text{page}=0$, $\text{offset}=3$ 'dür.
 - $\text{page_table}[0]$ 'ın değeri 5, $\text{page size}=4$ ise
 - $(\text{p_num} \times \text{p_size}) + \text{offset} \rightarrow 5 \times 4 + 3 = \mathbf{23}$
 - Mantıksal adres 4 için (e değeri):
 - $\text{page}=1$, $\text{offset}=0$ 'dır.
 - $\text{page_table}[1]$ 'ın değeri 6, $\text{page size}=4$ ise
 - $(\text{p_num} \times \text{p_size}) + \text{offset} \rightarrow 6 \times 4 + 0 = \mathbf{24}$



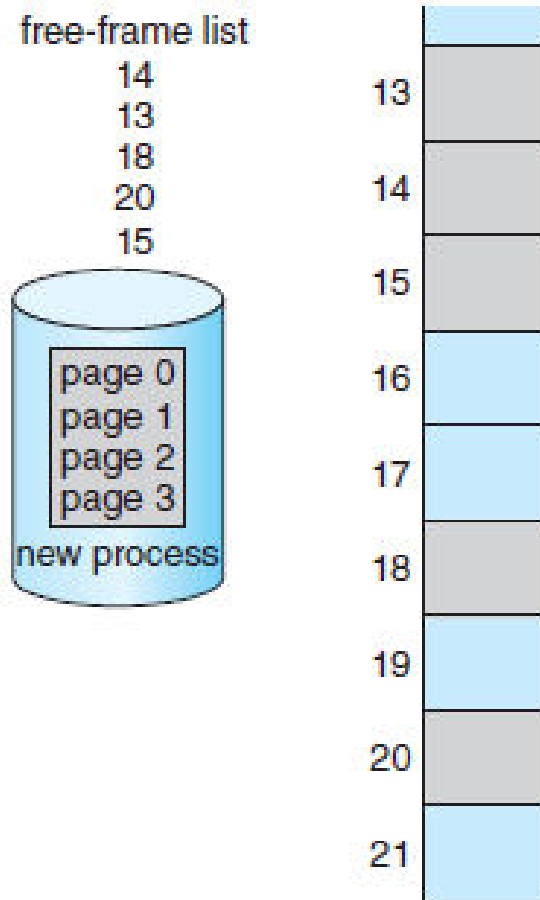
Sayfalı Bellek Tahsisi - Paging

- Bellek yönetiminde **paging** kullanıldığında, dışsal parçalanma olmaz.
- Ancak, içsel parçalanma olabilir.
 - Bir sürecin bellek gereksinimleri sayfa sınırlarıyla uyuşmazsa, son çerçeve tamamen dolu olmayabilir.
 - Örneğin, sayfa boyutu 2.048 byte olan bir sistemde;
 - 72.766 byte'lık bir işlem, 35 tam sayfa ve artı 1.086 byte'a ihtiyaç duyacaktır.
 - Ancak 36 çerçeve tahsis edilecek ve $2.048 - 1.086 = 962$ byte'lık içsel parçalanma olacaktır.
- İçsel parçalanmayı azaltmak için düşük sayfa boyutu tercih edilir.
 - Ancak bu durum, page-table girişini artıracığından ek yüke neden olur.
 - Sayfa boyutu arttıkça ek yük azalır.
 - Ayrıca Disc I/O durumlarında sayfa boyutu büyüdükçe verim artar
 - Süreçler, Veri boyutları ve Ana Bellek arttıkça sayfa boyutu artma eğilimindedir
 - Bugün Windows 10, 4 KB ve 2 MB sayfa boyutlarını desteklemektedir.
 - Linux, standart 4 KB ve **Huge Pages** olarak adlandırılan büyük boyutları destekler

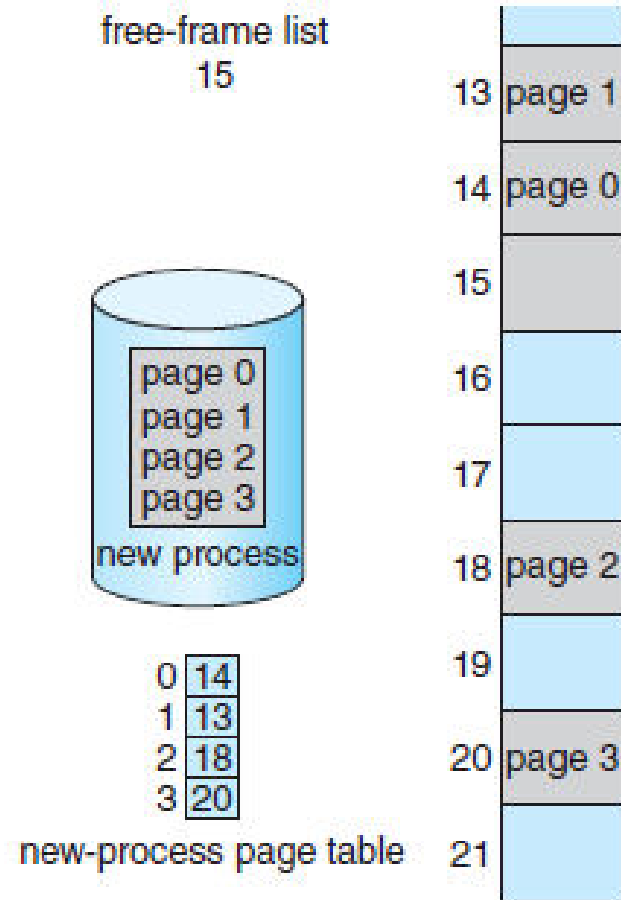
Sayfalı Bellek Tahsisi - Paging

- Yaygın olarak 32 bit bir CPU'da **page-table** girişleri 4 byte'dır.
 - 32 bitlik bir page table tanımı, 2^{32} 'lik bir adres uzayını ifade eder.
 - Çerçeve boyutu (page size) 4 KB (2^{12}) ise, bu sistem 2^{44} byte (16 TB), fiziksel belleği adresleyebilir.
 - Gerçekte 4 byte'lık tanımlara izin verilmez. Dolayısıyla daha az bir fiziksel bellek adreslenebilir.
- Sisteme bir süreç geldiğinde, page'lerle ifade etmek için boyutu incelenir.
 - Sürecin her page'i bir çerçeveye ihtiyaç duyar.
 - Bu nedenle süreç, n sayfa gerektiriyorsa, bellekte en az n çerçeve bulunmalıdır.
 - Eğer sistem uygunsa sürecin ilk sayfası tahsis edilen çerçevelerden birine yerleştirilir ve bu çerçeve süreç için page table'a yerleştirilir.
 - Devam eden sayfalar sonraki uygun çerçevelere yüklenerek süreç devam eder.
 - Paging'in en önemli yönü, belleğin programcı tarafından görünüşü ile gerçek fiziksel bellek arasında net bir ayırım yapmasıdır.
 - Programcı belleği yalnızca o anki süreç için tek parça olarak görür.

Sayfalı Bellek Tahsisi - Paging



(a) Tahsis öncesi



(b) Tahsis sonrası

Sayfalı Bellek Tahsisi - Paging

- İşletim sistemi fiziksel belleği yönettiği için fiziksel belleğin tahsis detaylarına hakim olmalıdır.
 - Hangi çerçevenin tahsis edildiği, hangi çerçevelerin boş olduğu vb.
 - Bu bilgiler genellikle **frame table** adı verilen tek bir veri yapısında tutulur.
 - Bu tabloda her fiziksel çerçeve için serbest mi tahsis edilmiş mi, tahsis edilmişse hangi süreç için tahsis edildiği gibi bilgiler yer alır.
- İşletim sistemi, süreçlere fiziksel adres verebilmesi için fiziksel adresleri mantıksal adreslerle eşlemelidir.
 - Bir kullanıcı bir sistem çağrısı üretirse (Örn. I/O) ve parametre olarak bir adres gönderirse, bu adresin doğru fiziksel adresle eşlenebilmesi gerekir.
 - İşletim sistemi komut kaydedicisinin (instruction register) ve register içeriklerinin bir kopyasını tuttuğu gibi page table'ında bir kopyasını tutar.
 - Bu kopya, işletim sisteminin mantıksal bir adresi fiziksel bir adresle eşlemesi gerektiğinde adresleri birbirine çevirmek için kullanılır.
 - Ayrıca CPU'ya bir süreç atanacağı zaman **dispatcher** tarafından da kullanılır.
 - Bu nedenle paging, context-switch süresini artırır.

Sayfalı Bellek Tahsisi - Donanım

- **Page Table**'lar, sürece özel veri yapıları olduğu için page table pointerları da PCB'de tutulan diğer register'lar gibi (örn. IP register) saklanırlar.
 - Süreçler yeniden planlandıklarında PCB ile birlikte page table'da yüklenir.
- Page table'ların donanım uyarlamaları, birkaç yolla yapılabilir.
 - En basit şekliyle page table, sayfa-adres dönüşümünü çok hızlı yapabilen bir dizi özel yüksek hızlı register'larla uygulanabilir.
 - Ancak bu register'ların her planlamada yer değiştirilmesi context-switch süresini artırır.
 - Page table küçükse (256 kayıt gibi), page table için register kullanmak makul olabilir.
 - Ancak çoğu modern CPU, çok büyük page table'ları (2^{20} kayıt) destekler.
 - Bu durumda register'ları kullanmak mümkün değildir.
 - Diğer bir yöntem olarak page table'lar ana bellekte tutulur ve bir **Page Table Base Register (PTBR)** page table'ı işaret eder.
 - CPU planlamada, page table'ın değiştirilmesi yerine sadece PTBR'nin değiştirilmesinden dolayı context-switch süresi azalır.

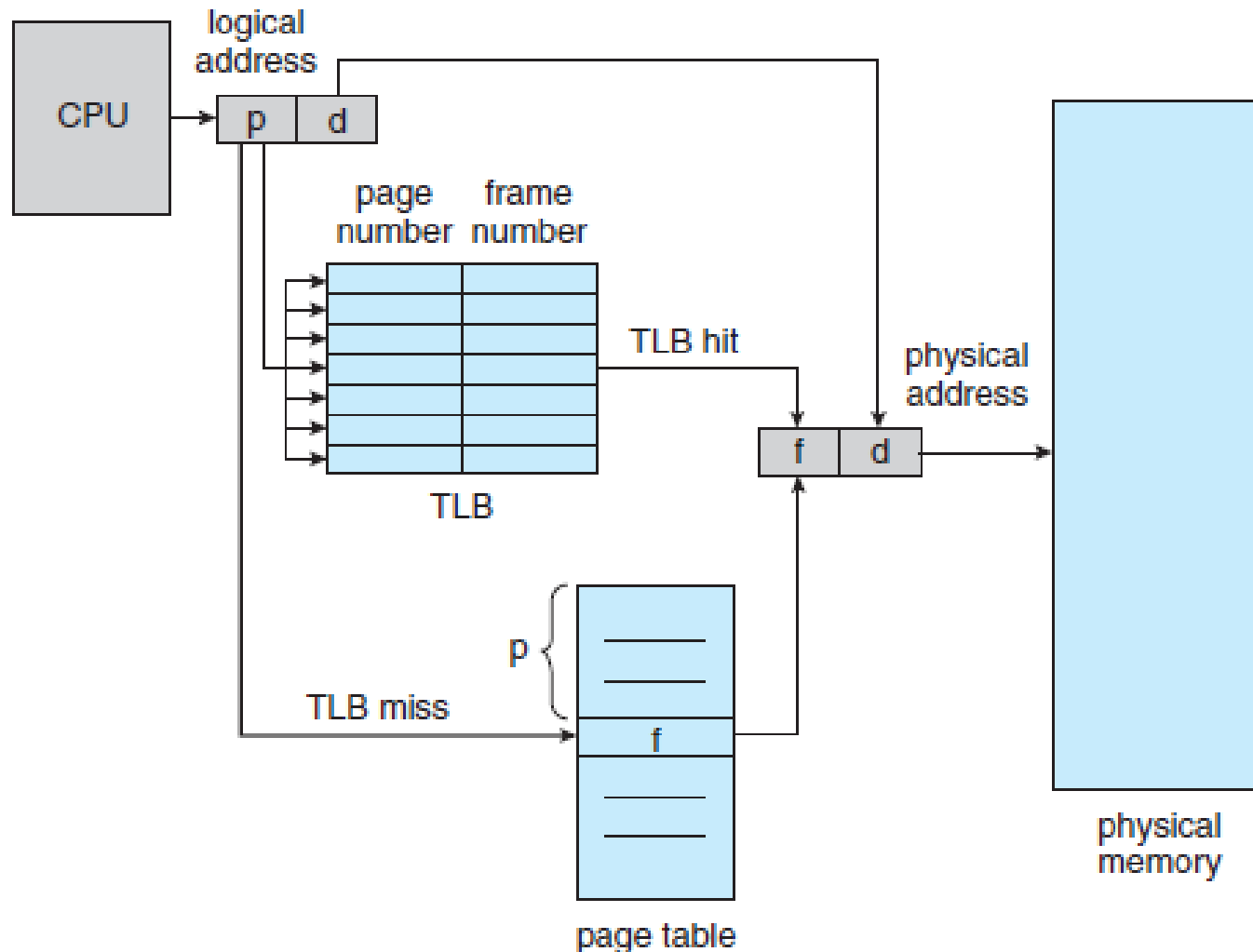
Sayfalı Bellek Tahsisi - TLB

- **Page Table**'ın ana bellekte tutulması context switch'i hızlandırırsa da bellek erişim süresi önemli bir sorundur.
 - i. konuma erişilmek istendiğini varsayarsak;
 - Öncelikle i. konum için PTBR (page table base pointer) kullanılarak i. page number için page table indekslenmeli ve frame number elde edilmelidir. **(1. bellek erişimi)**
 - Sonra frame number ve page offset birleştirilerek i'nin fiziksel adresine erişilir **(2. erişim)**
 - 1. Page table kayıtları, 2. Gerçek veriler. **2 kat gecikme!!!**
- Bunun çözümü için Etkin Sayfalar Önbelleği (**Translation Look-aside Buffer-TLB**) kullanılır.
 - TLB, yüksek hızlı bir önbellektir. Kayıtlar; Key – Value çifti ile tutulur,
 - CPU bir logical adres ürettiğinde, MMU önce page number'ı TLB'ye sorar.
 - Page number (key) varsa çerçeve numarasını (value) hemen kullanır ve belleğe erişir.
 - Yoksa (**TLB Miss**) önceki adımda değinildiği gibi ana bellekten sayfa numarasını çağırır.
 - Yeni page number ve frame bilgisi TLB'ye eklenir.
 - Bu değişim; en az kullanılan (least recently used - LRU), round-robin yada rastgele değiştirilebilir.

Sayfalı Bellek Tahsisi - TLB

- Bazı TLB'ler, her TLB kaydında Adres Uzayı Kimliği (**address-space identifier-ASIDs**) barındırır.
 - Bir ASID, her süreç için benzersizdir ve bu süreç için adres uzayı korunur.
 - Bir page number dönüşümü istendiğinde process ile ASID eşleşmediğinde girişim **TLB miss** olarak algılanır.
 - ASID TLB'nin korunmasını sağladığı gibi aynı anda birden fazla sürecin TLB'yi kullanabilmesini olanaklı kılar.
- TLB ayrı ASID'leri desteklemiyorsa, her context switch sürecinde bir sonraki yürütülen sürecin yanlış TLB'yi kullanmaması için TLB temizlenir (**TLB flush**)
 - Aksi takdirde TLB geçerli mantıksal bellek adresleri içeren ancak önceki sürece ait fiziksel adresleri dönüştürebilir. (Bkz. **Meltdown Attach**)

Sayfalı Bellek Tahsisi - TLB



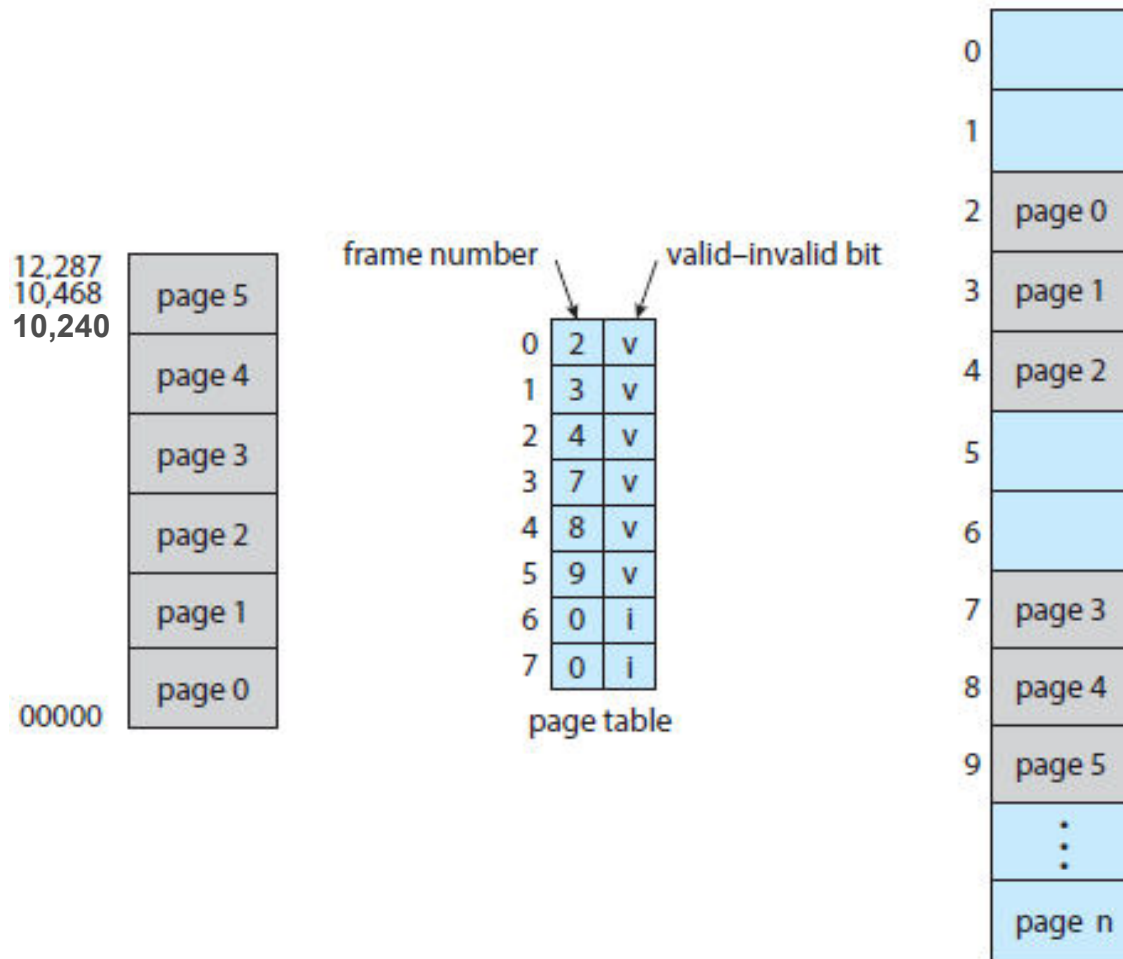
Sayfalı Bellek Tahsisi - TLB

- Erişilmek istenen sayfa numarasının TLB’de bulunma yüzdesine **hit ratio** denir
 - %80 hit ratio, istenen sayfa numarasının TLB’de %80 oranda olduğunu ifade eder.
 - Örn. belleğe erişimin 10 ns. olduğu varsayılın. Bu durumda;
 - Sayfa numarasının TLB’de olduğu durumda bellek erişimi 10 ns., yoksa; önce sayfa tablosu ve çerçeve numarası için belleğe erişileceği için 20 ns. sürer.
- Günümüzde CPU’lar birden fazla TLB seviyesi sağlayabilirler.
 - Örn. Intel Core i7 CPU;
 - 128 girişli L1 komut TLB’sine ve 64 girişli L1 veri TLB’sine sahiptir.
 - L1’de bir **TLB Miss** olması durumunda, ilave 6 CPU çevrimi gerektiren, 512 girişli L2 TLB’yi kullanır.
 - L2’de de TLB Miss durumu; yüzlerce ilave CPU çevrimi yada işletim sistemi kesmesi gerektiren çerçeve adresini bulmak için ana bellekteki page table’a erişim ile devam eder.
- Bu durum donanım özelliklerinin bellek performansı üzerinde önemli bir etkiye sahip olduğunu göstermektedir.

Sayfalı Bellek Tahsisi - Koruma

- Paging kullanan bir ortamda bellek koruması, her çerçeveye ilişkili page table'da tutulan koruma bitleri ile gerçekleştirilir.
 - Bir bit, bir page'i read–write yada read-only olarak tanımlayabilir.
- Belleğe yapılan her referans doğru sayfa numarasını bulmak için page table'dan geçer.
 - Fiziksel adresin hesaplanması sürecinde salt okunur bir page'e yazma yapılmadığını kontrol etmek için koruma bitleri kullanılır.
 - Salt okunur bir page'e yazmak istenildiğinde bir **hardware trap (memory-protection violation)** meydana gelir.
- Genel olarak page table'daki her kayda bir ek bit eklenir. (valid-invalid)
 - Bu bit, valid olarak ayarlandığında ilgili page, sürecin mantıksal adres uzayında kabul edilir ve bu erişim yasalıdır.

Sayfalı Bellek Tahsisi - Koruma



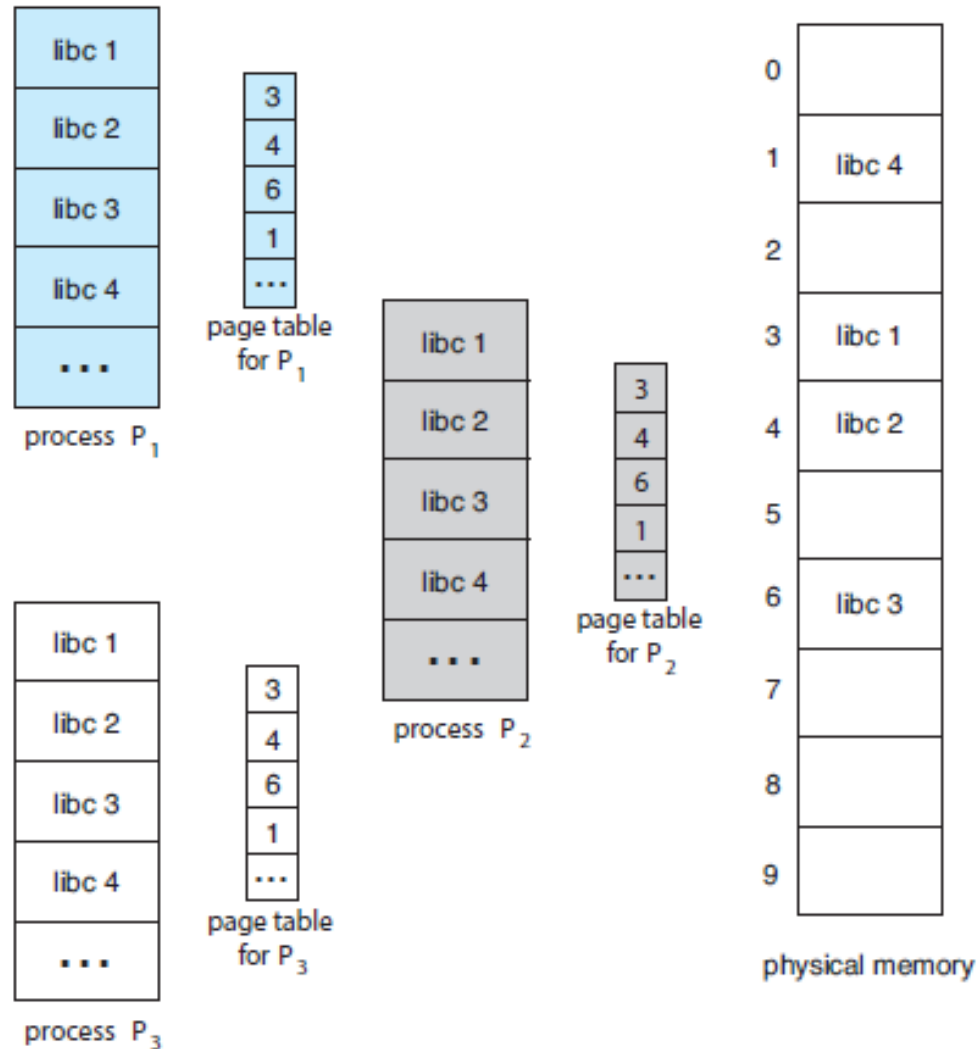
Sayfalı Bellek Tahsisi - Koruma

- Nadiren bir süreç sahip olduğu tüm bellek uzayını kullanır.
 - Çoğu süreç kullanabilecekleri adres uzayının küçük bir bölümünü kullanır.
 - Dolayısıyla page table'ın çoğu kullanılmaz, ancak değerli bellek israf edilir.
 - Bazı sistemler page table'ın boyutunu belirlemek için sayfa tablosu uzunluk kaydedicisi (**page-table length register - PTLR**) donanımı sağlar.
 - Bu değer, talep edilen adresin süreç için geçerli aralıkta olup/olmadığını doğrulamak için kullanılır.
 - Uygun aralıkta değilse bu durum işletim sistemine bir hata olarak bildirilir.

Sayfalı Bellek Tahsisi-Shared Paging

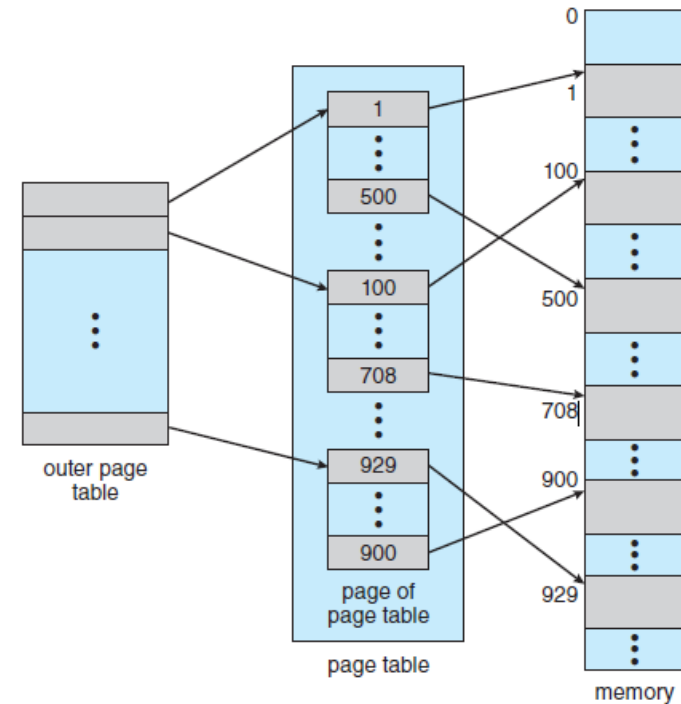
- Paging'in bir avantajı da birden çok sürecin ortak kodları paylaşma olasılığıdır.
 - Unix ve Linux'ün bir çok sürümü sistem çağrısı arabirimi olarak C kütüphanelerini kullanır.
 - Örneğin tipik bir Linux sisteminde çoğu kullanıcı süreci, standart C kütüphanesi olan libc'yi kullanır.
 - Kötü bir seçenek her sürecin kendi libc kütüphanesini kendi adres uzayına yüklemesidir.
 - Bir sistemde 40 süreç varsa bu durumda 2 MB libc toplam 80 MB bellek harcar.
 - Kod evrensel (**reentrant code**) bir kods, bu kütüphaneler paylaşılabilir.
 - Bu durumda libc kütüphanesinin yalnızca bir kopyasının fiziksel bellekte tutulması yeterlidir.
 - Yani her süreç'in page table'ı libc'nin aynı fiziksel belleğiyle (frame) eşlenir.
 - Ayrıca libc gibi çalışma zamanı kütüphanelerine ek olarak, yoğun olarak kullanılan compiler, grafik arabirimleri (pencereler vs.), veritabanı gibi sistemler de paylaşılabilir.
 - Bir sistemdeki süreçler arasında belleğin paylaşılması, Bölüm 4'te sunulan thread'lerin bir task'ın adres uzayını paylaşmasına benzer.
 - Bazı işletim sistemleri Bölüm 3'te anlatılan ve bir IPC tekniği olan shared-memory yöntemini shared paging ile uygular.

Sayfalı Bellek Tahsisi-Shared Paging



Page Table Yapısı

- **Hiyerarşik Sayfalama:** Çoğu modern bilgisayar sistemi çok büyük mantıksal adres uzayını destekler (2^{32} ila 2^{64}). **Çok Büyük page table!**
 - Örn. 32 bit mantıksal adres uzayına sahip bir sistemde page-size 4KB (2^{12}) ise bir page table 1 milyondan fazla kayıt içerebilir ($2^{20}=2^{32}/2^{12}$).
 - Her bir giriş 4 byte olsa, her bir süreç tek başına page table için 4 MB'a kadar fiziksel belleğe ihtiyaç duyabilecektir. [Bkz ☺](#)
 - Böyle bir kaydın, bitişik bellek tahsisi ile bellekte tutulması istenmez.
 - Bu sorunun bir çözümü page-table'ı da paging'li (iki seviyeli) şekilde bellekte tutmaktır.
 - Örn. 32 bit mantıksal adres uzayına ve 4KB page-size'a sahip sistem için; (4B page-table girişi için)
 - Mantıksal bir adres 20 bitlik page-number ve 12 bitlik page-offset değerine sahiptir.
 - Page-table yeniden paging yapıldığı için page-number ayrıca 10 bitlik bir page-number'a ve 10 bitlik bir page-offset'e ayrılmıştır.

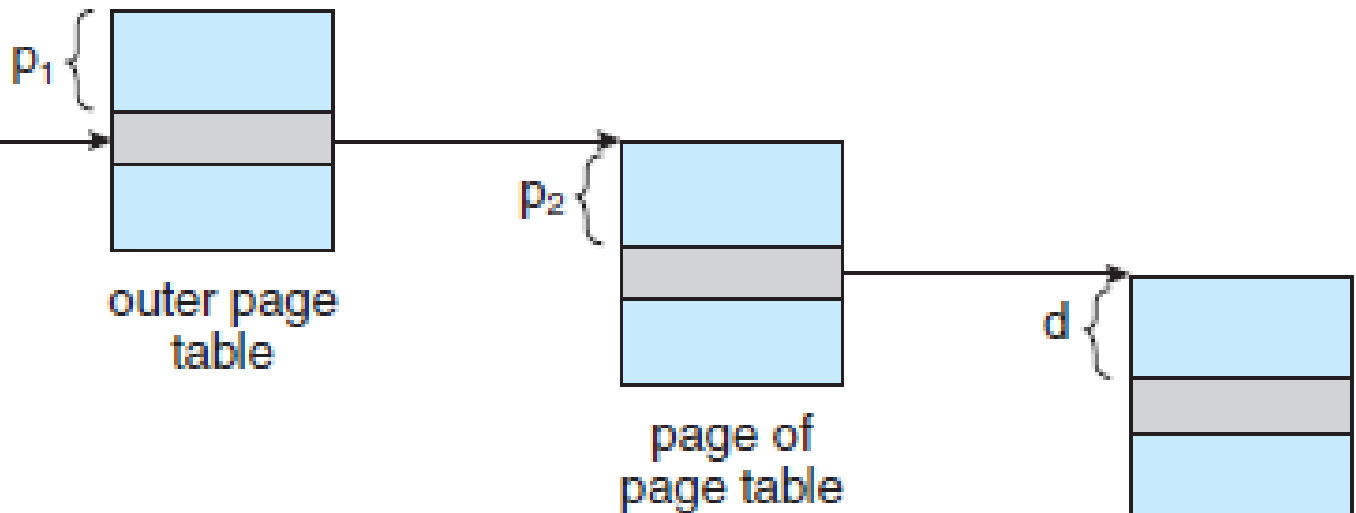


page number		page offset
p_1	p_2	d
10	10	12

Page Table Yapısı - Hiyerarşik

- Burada P_1 , dış sayfa tablosunun (outer page table) indisi, P_2 ise iç sayfa tablosunun (inner) yeni paging içindeki indisini ifade eder.
 - Adres çevirisi dış sayfa tablosundan içe doğru çalıştığı için, bu şema ileriye dönük bir sayfa tablosu (**forward-mapped page table**) olarak da bilinir.

logical address



Page Table Yapısı - Hiyerarşik

- 64 bit mantıksal adres uzayına sahip bir sistem iki seviyeli bir paging'e uygun değildir.
 - Böyle bir sistemde page-size'ın 4 KB (2^{12}) olduğunu varsayalım.

- Bu durumda, page-table en fazla 2^{52} girişten oluşur.
- İki seviyeli bir sayfalama şeması kullanılırsa, iç sayfa tablosu uygun bir sayfa uzunluğunda olabilir ve 2^{10} adet 4 baytlık giriş içerebilir. ($2^{10} * 2^2 = 2^{12} = 4\text{KB}$ uygun page-size)

outer page	inner page	offset
p_1	p_2	d
42	10	12

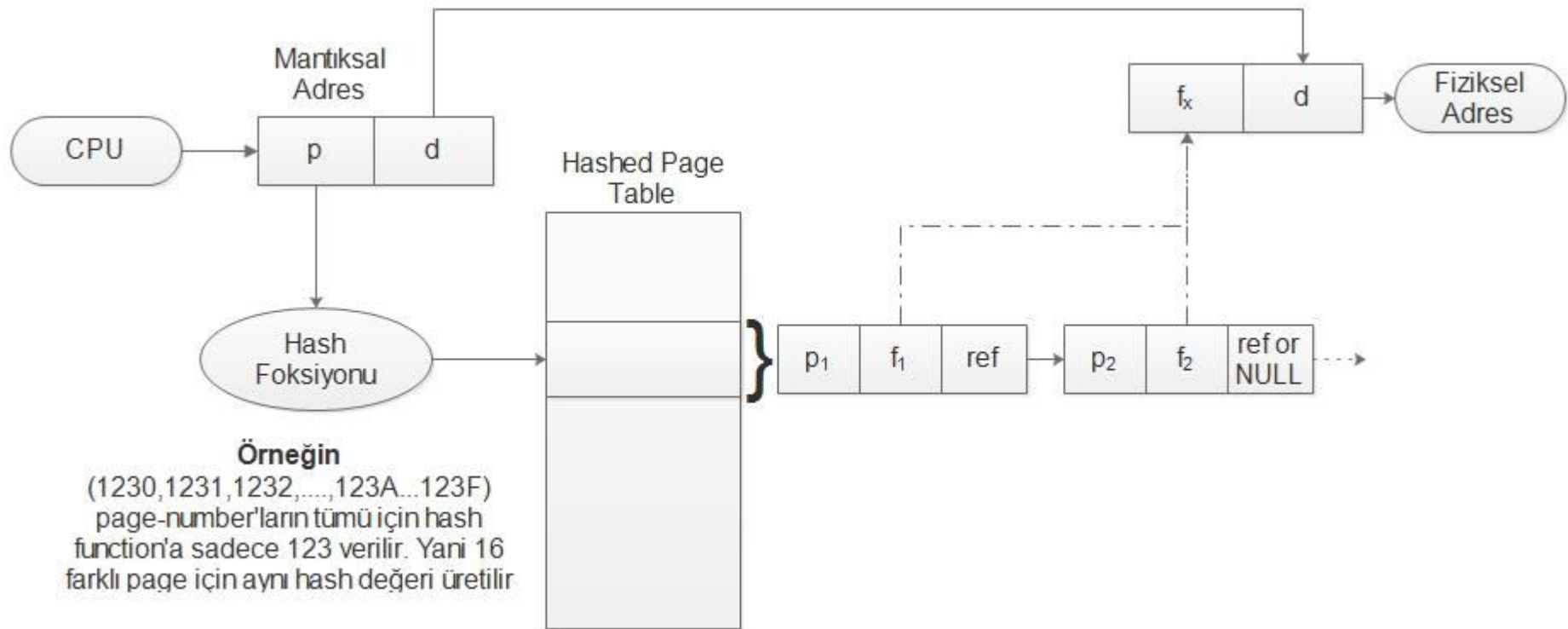
- Dış sayfa tablosu 2^{42} girişten ve 2^{44} bayttan oluşur.
- Böylesine büyük page-table'dan kaçınmanın yolu, dış sayfa tablosunu daha küçük parçalara bölmektir. (Bu yaklaşım, esneklik ve verimlilik için bazı 32 bit CPU'larda da kullanılır.)
- Dış page-table çeşitli şekillerde bölünebilir. Örneğin, dış page-table'ı paging'leyerek üç seviyeli bir sayfalama elde edebiliriz.
- 64 bitlik bir adres alanı için dış page-table hala 2^{34} bayt (16 GB) boyutundadır. Çok büyük!
- Bu süreç 3., 4. seviye şeklinde devam eder. 64-bitlik UltraSPARC 7 seviye kullanır.

2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12

Page Table Yapısı - Hashed Page

- **Hashed page table**, 32 bitten büyük adres uzaylarına yönelik bir yaklaşımdır.
 - Hash değerinin, page number olarak kullanıldığı karma bir page-table'dır.
 - Hash tablosundaki her kayıt, bir linked-list ile ilişkilidir.
 - Her öge üç alandan oluşur: (1) page number, (2) page frame ve (3) linked-list'deki sonraki ögeyi işaret eden bir pointer.
- Algoritma şu şekilde çalışır:
 - Mantıksal adresteki page number hash table içerisine hashlenir.
 - Page number, linked-list'in ilk ögesindeki field-1 ile karşılaştırılır.
 - Bir eşleşme varsa, fiziksel adresi oluşturmak için page-frame (field-2) kullanılır.
 - Yoksa, linked-list'deki sonraki kayıtlarla eşleşen bir page number aranır.
- 64 bitlik adresler için hashed table'ın bir varyasyonu olan **clustered page tables** kullanılır.
 - Bu modelde tek bir page yerine, daha fazla page (16 page) referans edilir.

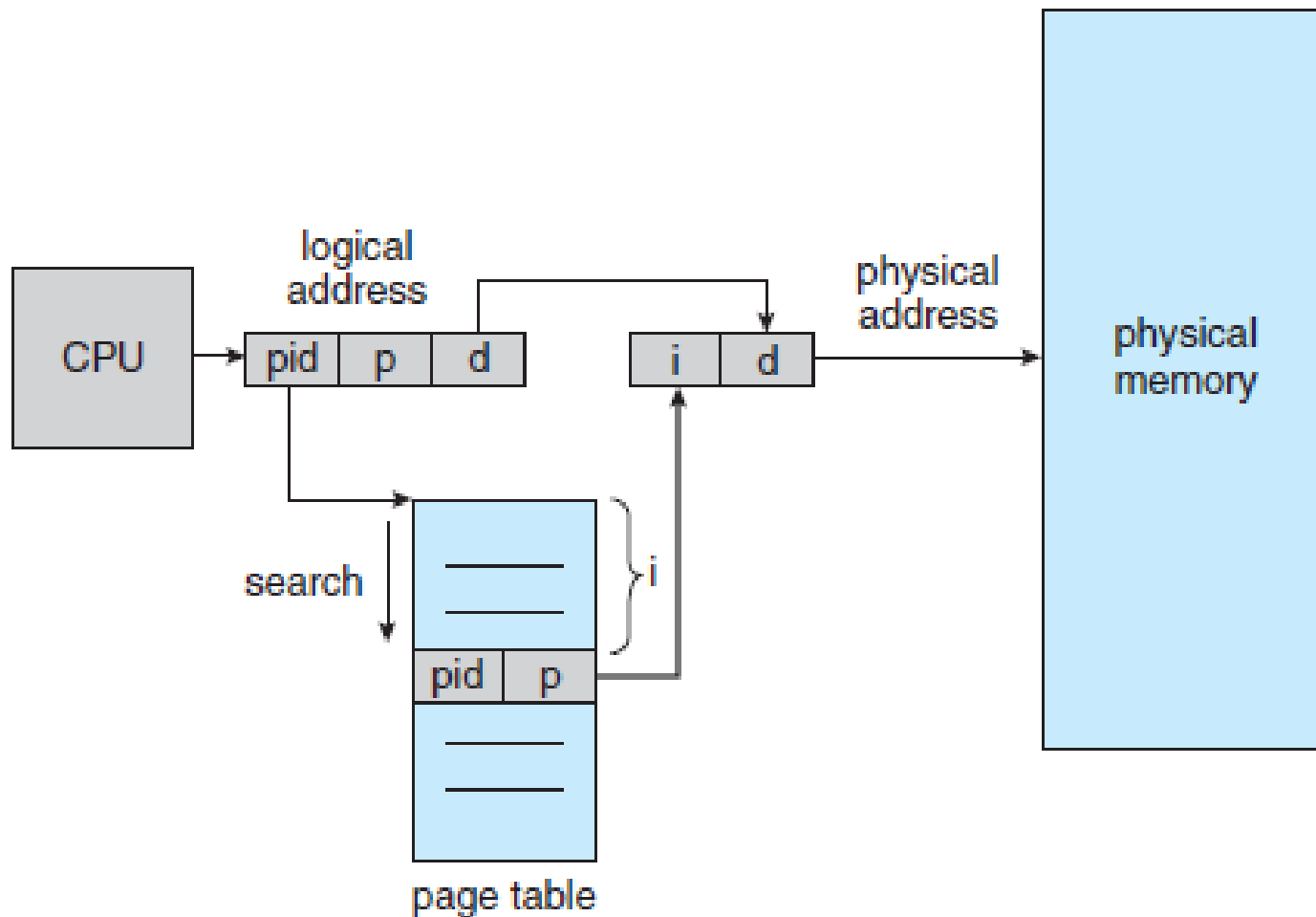
Page Table Yapısı - Hashed Page



Page Table Yapısı – Inverted Page Tables

- Her sürecin bir page-table'ı vardır (İlk iki yapı için).
 - İşletim sistemi mantıksal adresleri page-table üzerinden fiziksel adrese çevirir.
 - Bu tablolar mantıksal adreslere (page-number) göre sıralandığından aramak kolaydır
 - Ancak bu tablolar, büyük miktarda fiziksel bellek tüketebilir. (çok fazla kayıt)
- Bu sorunu çözmek için önerilen yöntemlerden biri de **Inverted Page Table**'dir.
 - Her sürecin bir page-table'ı olması ve olası tüm mantıksal page'lerin kaydını tutması yerine, tüm fiziksel page'ler (frame) için bir kayıt tutar.
 - Her giriş, page'in sahibi sürecin bilgileri yanında gerçek bellek konumunda depolanan page'in mantıksal adresini barındırır.
 - Bu nedenle sistemde yalnızca bir page-table vardır.
 - IPT, bu nedenle page-table'ın her kaydı için bir adres alanı tanımlayıcısı (**ASID**) içerir.
 - Tablo boyu azalsa da tüm tabloyu tarama/arama süresini artırır.
 - Ayrıca shared page yapısı kullanılamaz. (ASID yada ProcessID ile ilişkilendirilmesi)
 - IPT'yi kullanan sistemler arasında 64-bit UltraSPARC ve PowerPC bulunur.

Page Table Yapısı – Inverted Page Tables

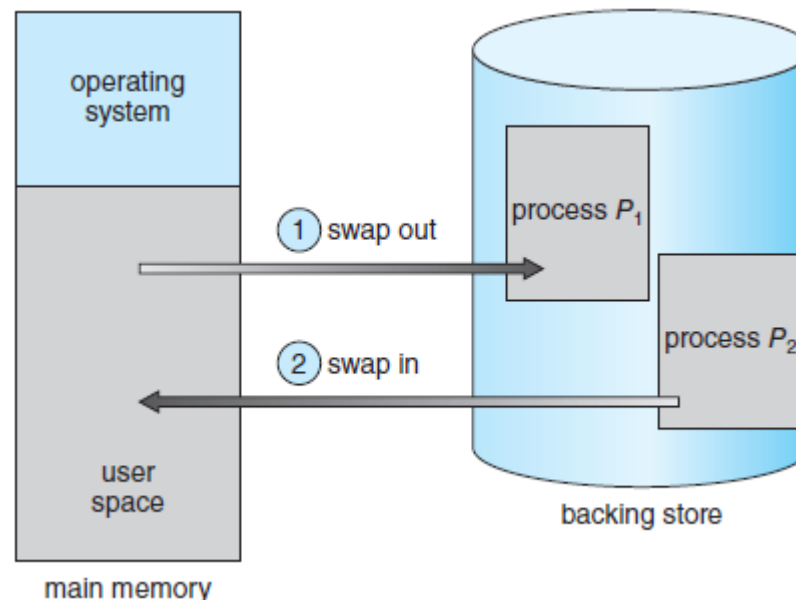


Page Table Yapısı – Oracle SPARC Solaris

- 64 bit SparcCPU üzerinde çalışan Solaris birden çok page table düzeyini tutarak tüm fiziksel belleği kullanmadan mantıksal bellek sorununu çözmeye odaklanır.
 - Temelde hash'lenmiş page table kullanır.
 - İki tane hash table kullanır. 1) Çekirdek, 2) Kullanıcı süreçleri için
 - Her biri mantıksal bellek adreslerini, fiziksel bellek adreslerine çevirir
 - Her bir hash table kaydı, haritalanmış sanal belleğin bitişik bir alanını temsil eder.
 - Bu durum her page için ayrı bir hash table kaydına sahip olmaktan daha verimlidir.
 - Her kaydın bir base adresi ve kaydın temsil ettiği page sayısını temsil eden bir aralık vardır.
 - Her adres için bir hash table araması gerekirse, fiziksel adrese çevirim çok uzun sürer.
 - Bu nedenle CPU, hızlı donanım aramaları için **Translation Table Entries (TTEs)** içeren bir TLB uygular.
 - Bu TTE'ler, en son erişilen page başına bir kayıt içeren bir **Translation Storage Buffer'da (TSB)** bulunur.
 - Bir Mantıksal Bellek Adresi geldiğinde; fiziksel adres dönüşümü için öncelikle donanım TLB'yi arar.
 - Bulunamazsa donanım aramaya neden olan mantıksal bellek adresine karşılık gelen TSB'de arar. (TTE arar)
 - Bulunursa bu kayıt TLB'ye aktarılır. Bulunamazsa kesme üretilir ve hash table'da arama yapılır.
 - Hash table'dan bulunan kayıttan TTE oluşturulur ve CPU bellek yönetim birimi tarafından TLB'ye yüklenmek üzere TSB'de saklanır.
 - Son olarak interrupt handler adres çevrimi tamamladıktan sonra kontrolü MMU'ya bırakır.

Takaslama - Swapping

- Süreçlere ait komutlar ve kullandıkları veriler, yürütülebilmek için ana bellekte olmalıdır.
- Ancak bir süreç veya bir bölümü, geçici olarak ana bellekten bir yardımcı bellek (**backing store**) birimine daha sonra geri alınmak üzere takaslanabilir (**swapping**).
- Takaslama, çalışan tüm süreçlerin ihtiyaç duyduğu toplam fiziksel belleğin, sistemin sunduğu gerçek fiziksel belleğin üzerinde olmasını mümkün kılar.



Takaslama - Swapping

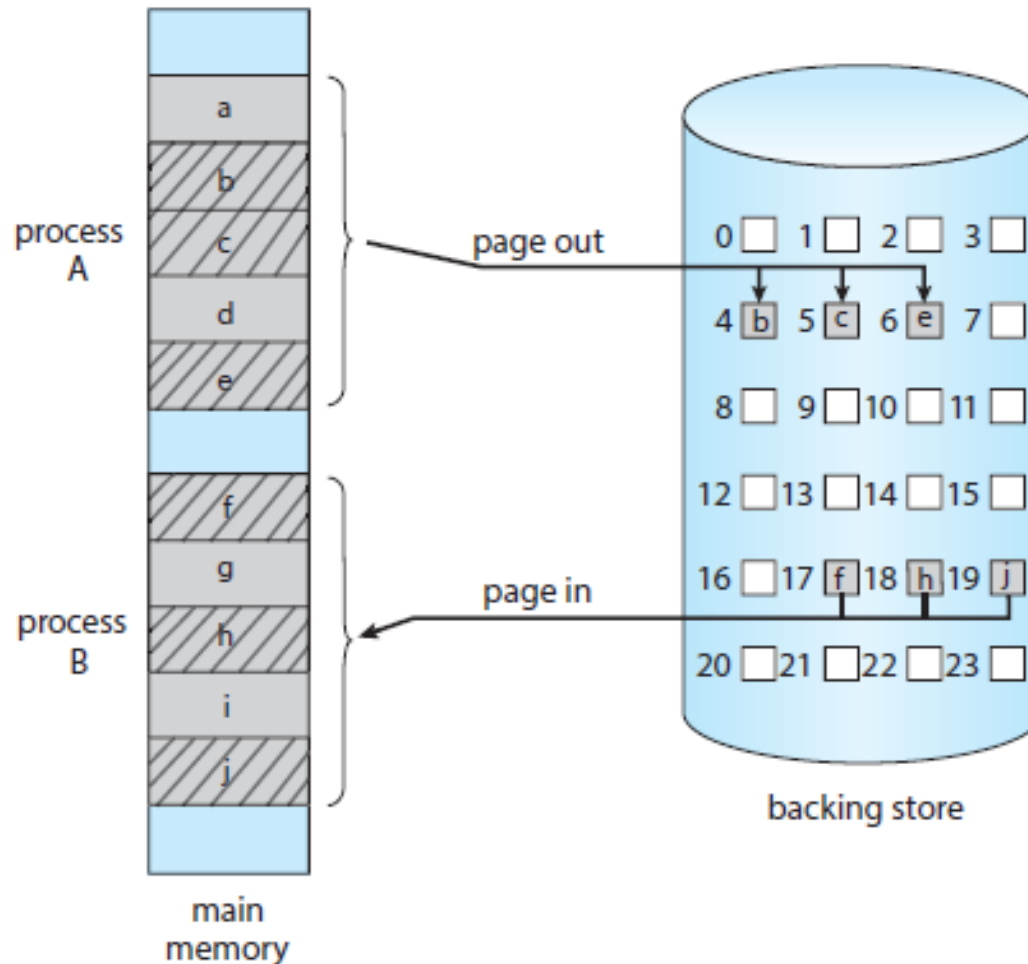
- **Standart Takaslama:** Tüm süreçlerin ana bellek ile bir yardımcı bellek arasında taşınmasını içerir.
 - Yardımcı bellek deposu genellikle hızlı bir ikincil diskidir.
 - Bu alanın, depolanması ve geri alınması gereken süreçlerin tümünü barındıracak kadar büyük ve doğrudan erişim imkanı olmalıdır.
 - Bir süreç veya parçası yardımcı belleğe takaslandığında, süreçle ilişkili veri yapıları da yardımcı belleğe yazılmalıdır.
 - Çok iş parçacıklı bir süreç için tüm iş parçacıklarına ait veri yapıları ile takaslanmalıdır.
 - İşletim sistemi ayrıca, takaslanan süreçler için meta verileri korumalıdır.
 - Bu yöntemin en önemli avantajı, fiziksel belleğin kapasitesinin çok üzerindeki taleplere izin vermesidir.
 - Boşta veya çoğunlukla boşta olan süreçler takas için tercih edilir.
 - Bu durum etkin olmayan süreçlere tahsis edilen herhangi bir belleğin aktif süreçlere ayrılabilmesini sağlar.
 - Takas edilen (**swap out**) aktif olmayan bir süreç tekrar aktif hale gelirse, ana belleğe yeniden takaslanır (**swap in**).

Takaslama - Swapping

- **Page'leri Takaslama:** Standart takaslama geleneksel UNIX sistemlerinde kullanılıyordu. Ancak artık çağdaş işletim sistemlerinde kullanılmamaktadır.
 - Çünkü tüm süreçleri ana bellek ile yardımcı bellek arasında taşımak için gereken süre çok fazladır. (Solaris, kullanılabilir belleğin çok az olduğu koşullarda kullanır)
 - Linux ve Windows da dahil olmak üzere çoğu sistem, artık bir sürecin sayfalarının (tüm süreci değil) takaslanmasını kullanır.
 - Bu strateji, fiziksel belleğin üzerinde talebin karşılanmasını yine de sağlar.
 - Ancak takaslamaya daha az sayıda page dahil olacağından, tüm süreçlerin takaslanması maliyetine neden olmaz.
 - Aslında, takaslama terimi artık genellikle standart takaslama anlamına gelir ve paging, page'leri takaslamayı ifade eder.
 - Bir **page out** işlemi, bir page'i bellekten yardımcı belleğe taşır. Ters, **page in** olarak adlandırılır.

Takaslama - Swapping

- Page'leri Takaslama:



Takaslama - Swapping

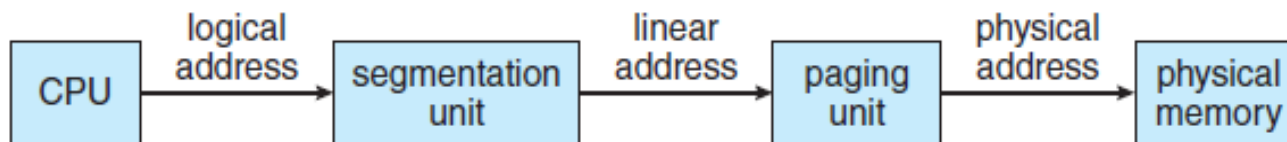
- **Mobil Sistemlerde Takaslama:** PC ve Server işletim sistemlerinin aksine, mobil sistemler takaslamayı desteklemez.
 - Mobil cihazlar, kalıcı depolama için genellikle sabit diskler yerine flash kullanırlar.
 - Ortaya çıkan disk kısıtı, mobil işletim sistemi tasarımcılarının takaslamadan kaçınmasının bir nedenidir.
 - Diğer bir nedeni, flash belleğin sınırlı sayıda yazma limiti ve bu cihazlarda, ana bellek ile flash bellek arasındaki zayıf verim sıralanabilir.
 - Takaslama yerine iOS, boş bellek belirli bir eşiğin altına düştüğünde, uygulamalardan belleği serbest bırakmasını ister.
 - Salt okunur veriler (kod gibi) ana bellekten kaldırılır ve daha sonra gerekirse flash bellekten yeniden yüklenir.
 - Değiştirilmiş veriler (yığın gibi) kaldırılmaz. Ancak, yeteri kadar bellek boşaltılamazsa uygulamalar sonlandırılabilir.
 - Android, iOS ile benzer bir strateji izler.
 - Yeterli boş hafıza yoksa bir süreci sonlandırabilir.
 - Ancak, bir işlemi sonlandırmadan önce uygulama durumunu flash belleğe yazar.

Intel 32 ve 64-bit Mimarileri

- Intel mimarisi, kişisel bilgisayar ortamlarına uzun yıllardır hakimdir.
- 16-bit Intel 8086 ve 8088, 1970'lerin sonunda ortaya çıktı ve kısa bir süre sonra, IBM PC'lerde kullanılan mikroçip olmasıyla dikkat çekti.
- Intel daha sonra 32-bit Pentium işlemci ailesini içeren bir dizi yonga (IA-32) üretti.
- Yakın zamanlarda Intel, x86-64 mimarisine dayalı bir dizi 64-bit yonga üretti.
- Şu anda, Windows, macOS ve Linux dahil olmak üzere en popüler PC işletim sistemlerinin çoğu Intel yongaları üzerinde çalışmaktadır.
 - [Linux başka mimarilerde de çalışmaktadır.](#)
- Ancak Intel, ARM'ın önemli bir başarıya sahip olduğu mobil sistemlerde hakimiyet sağlayamamıştır.

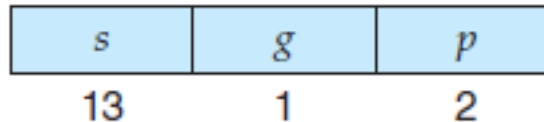
Intel IA-32

- IA-32 sistemlerinde bellek yönetimi iki bileşene ayrılır: **Segmentation** ve **Paging**.
 - Segmentation 64 bit sistemlerde kullanılmaz ancak 32 bit uygulamalar için desteği devam etmektedir.
- CPU **mantıksal adresleri** üretir ve segmentation birimine aktarır.
- Segmentasyon birimi, her mantıksal adres için bir **lineer adres** üretir.
- Lineer adres daha sonra paging birimine verilir ve bu da ana bellekteki **fiziksel adresi** oluşturur.
- Yani IA32’de, **segmentation** ve **paging** birimleri, bellek yönetim birimini (MMU) oluşturur.



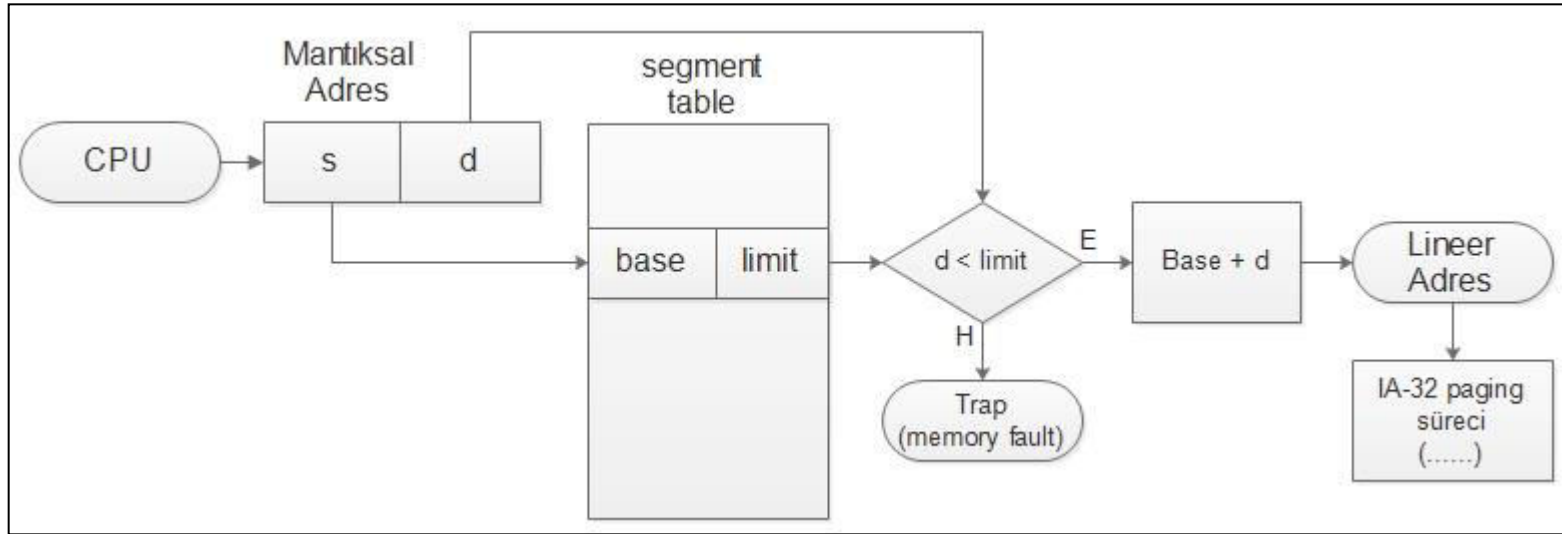
Intel IA-32 - Segmentation

- IA-32 mimarisinde, bir segment en fazla 4 GB olabilir. Süreç başına maksimum segment sayısı 16K'dır. Bir sürecin mantıksal adres alanı iki bölüme ayrılmıştır.
 - İlk bölüm, sürece özel 8K'ye kadar olan segment'lerden oluşur.
 - İlk bölüm, yerel tanımlayıcı tablosunda (**local descriptor table - LDT**) tutulur.
 - İkinci bölüm, tüm süreçler arasında paylaşılan ve 8K olabilen segmentlerdir.
 - İkinci bölüm, genel tanımlayıcı tablosunda (**global descriptor table - GDT**) tutulur.
 - LDT ve GDT'deki her kayıt, o segmentin base konumu ve limiti dahil olmak üzere ayrıntılı bilgi içeren 8 baytlık bir segment tanımlayıcısından oluşur.
 - **Mantıksal adres**, selector'ün 16 bit, ofset'in 32 bit olduğu bir adres çiftidir (**selector, ofset**)



- **Selector için;** s, segment numarasını, g segmentin GDT'de mi yoksa LDT'de mi olduğunu gösterir. ve p korumayla ilgilidir.
- **Ofset**, söz konusu byte'ın segment içindeki konumunu belirten 32 bitlik bir sayıdır.

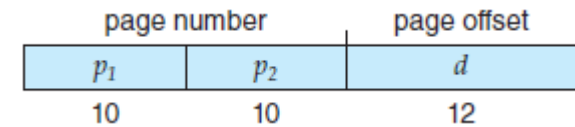
Intel IA-32 - Segmentation



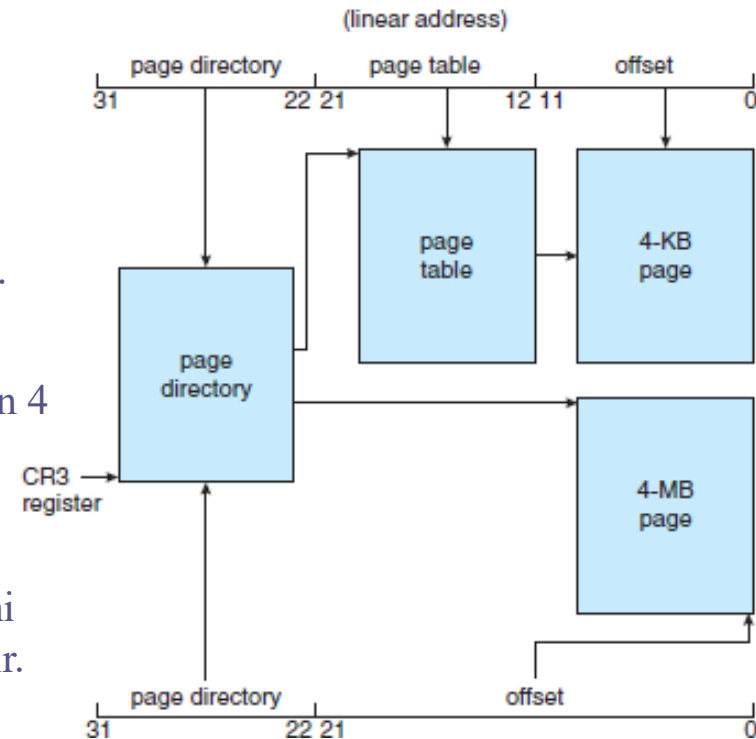
- İlgili segmentteki Base ve Limit bilgileri lineer adresi oluşturmak için kullanılır.
- Limit değeri kullanılarak adresin geçerliliği kontrol edilir.
- Adres geçerli değilse;
 - İşletim sisteminde bellek hatası (memory fault) oluşur.
- Geçerliyse;
 - offset, base'e eklenir, ve böylece lineer adres elde edilir.
- **Lineer adres 32 bittir.**

Intel IA-32 - Paging

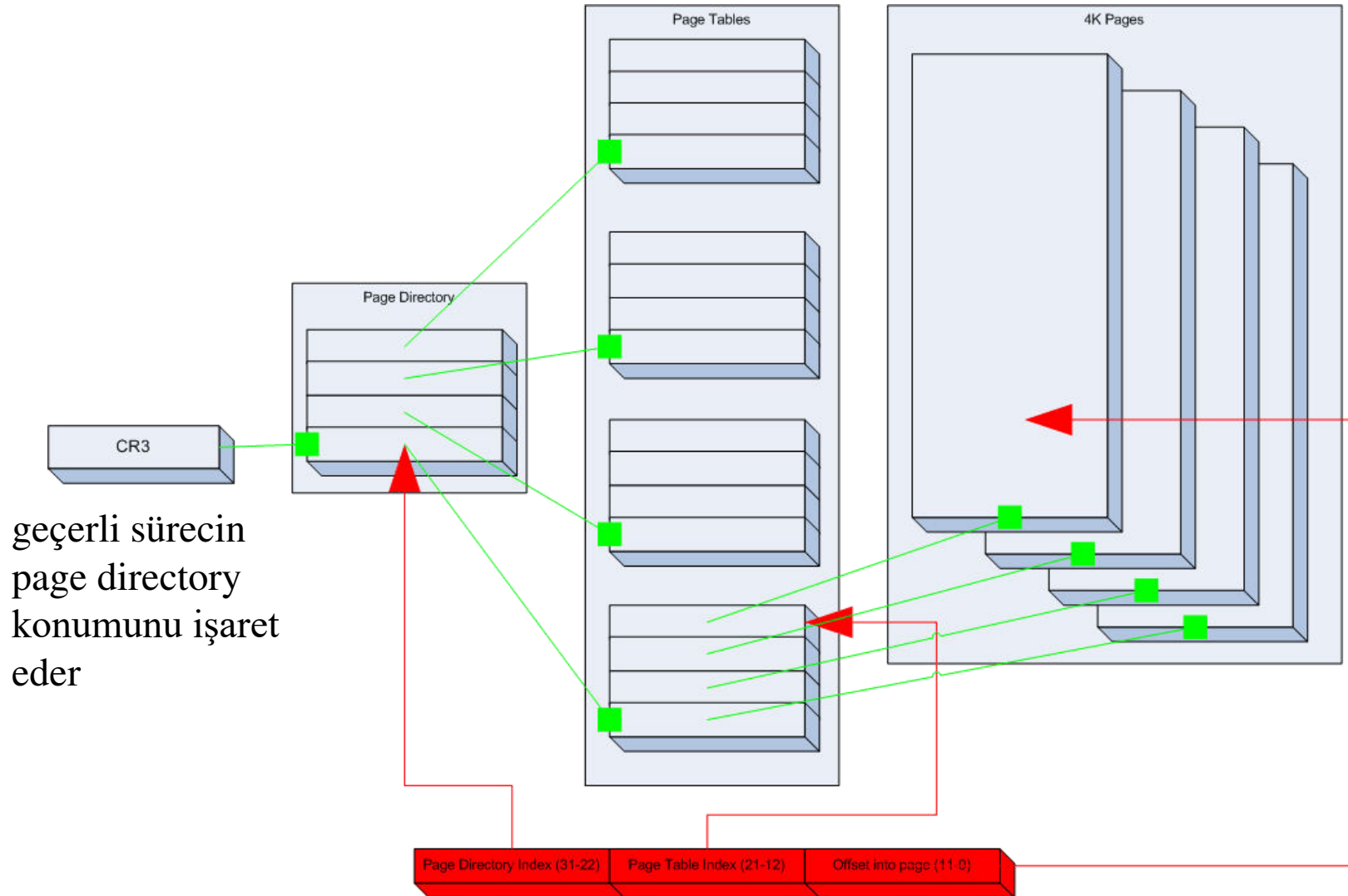
- IA-32, 4KB veya 4MB page-size'a izin verir.



- 4 KB'lik page'lerde, lineer adresin bölümlenmesinde iki seviyeli bir paging kullanır.
- Yüksek değerlikli 10 bit, page directory'i, en dıştaki page-table kaydını işaret eder
 - CR3 register'ı, geçerli sürecin page directory'ni işaret eder.
 - Page directory kaydı, lineer adresteki 10 bit'in tanımladığı bir iç page table'ı işaret eder.
 - En düşük 12 bit, page-table'ın offset'ini tanımlar.
 - Page directory'deki bir kayıt, Page-Size bayrağıdır. (Default 4KB, ayarlanmışsa 4MB).
 - Bayrak set edilirse düşük değerlikli 22 bit doğrudan 4 MB'lık page table'ın offset'ini tanımlar.
 - Page-table'lar diske swap'lenebilir.
 - Bu durumda page-directory, page-table'ın diskte mi yoksa bellekte mi olduğunu gösteren bir bit kullanır.

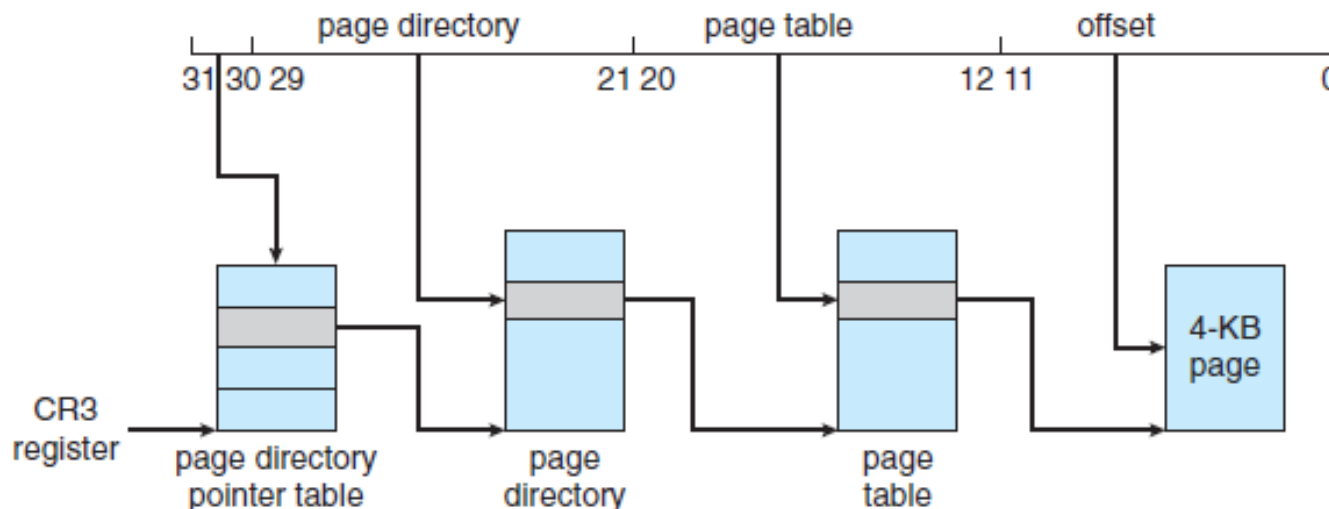


Intel IA-32 - Paging

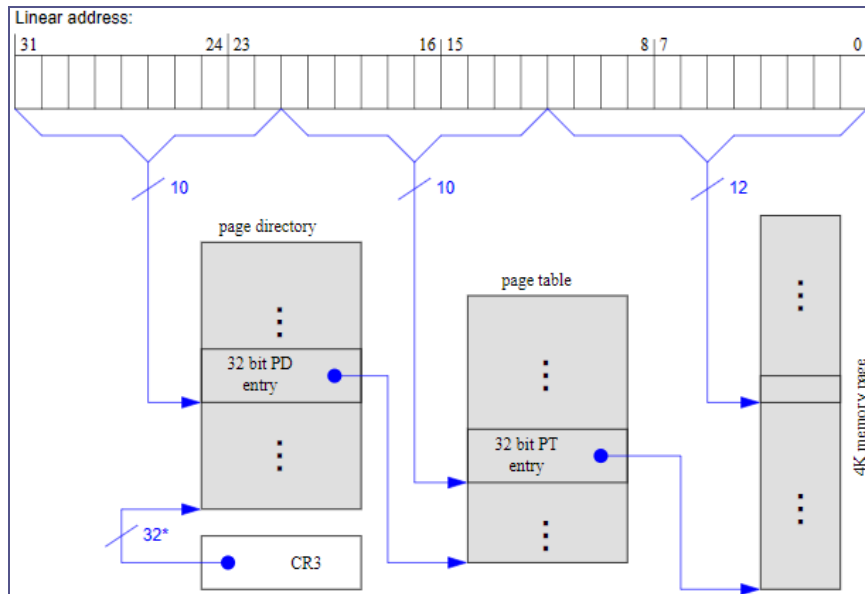


Intel IA-32 - Paging

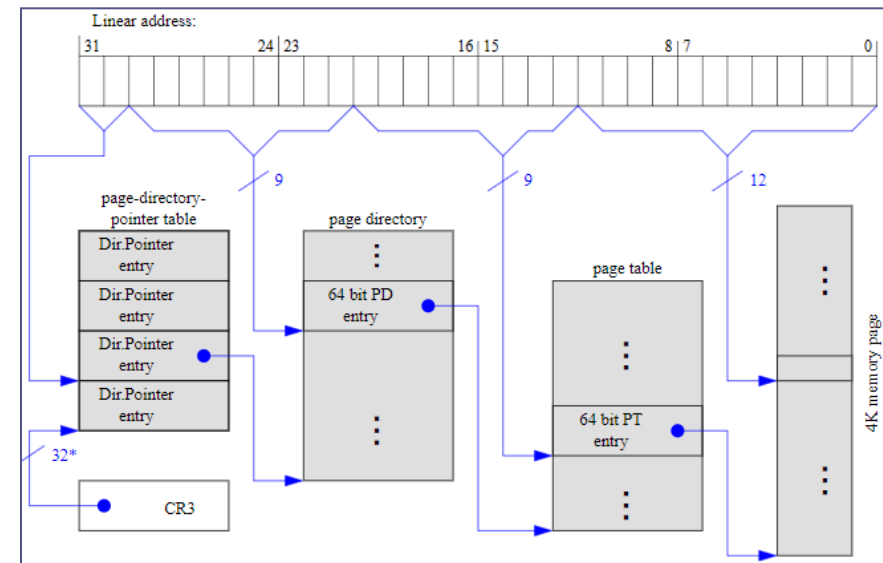
- Intel 4GB bellek sınırını genişletmek için **physical address extension (PAE)** olarak isimlendirilen ve 3 seviyeli bir paging yöntemine geçmiştir.
 - PAE, page-directory ve page-table girişlerini 32 bitten 64 bit'e çıkarmıştır.
 - Bu page-table'ların ve page-frame'lerin Base adresinin **20'den 24 bit'e** uzamasını sağladı.
 - IA-32 PAE desteği, adres alanını 64 GB'a kadar yükseltmiştir. (Fiziksel adres $24+12=36$ bit)
 - Linux ve MacOS PAE'yi destekler. Ancak Windows'un 32 bit sürümlerinde PAE etkin olsa da 4 GB adres sınırı vardır.
 - Lineer adresteki yüksek değerlikli 2 bit page directory pointer table için kullanılır.



Intel IA-32 - Paging



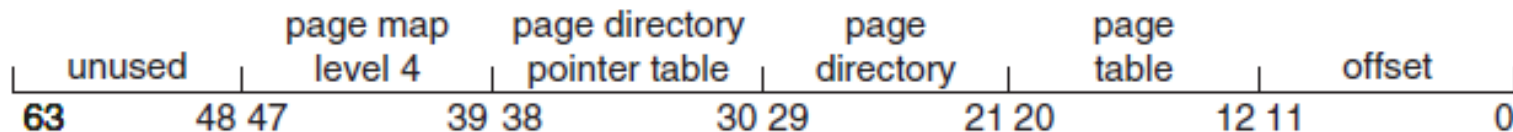
PAE's 4KB page-size



PAE ile 4KB page-size

Intel x86-64

- Intel'in, 64bit mimariler için ilk tasarımı IA-64 (**Itanium**)'tü. (Yaygınlaşmadı)
- AMD ise mevcut IA-32'ye dayanan 64 bitlik bir mimari geliştirmeye başladı.
 - X86-64, çok daha büyük mantıksal ve fiziksel adres alanlarını destekledi.
 - Geçmişte AMD, genellikle Intel'in mimarisine dayalı çipler geliştirmişti, ancak şimdi Intel, AMD'nin x86-64 mimarisini benimsedi ve roller tersine döndü.
 - Bu mimari, **AMD64** ve **Intel64** yerine, genel bir terim olan **x86-64** olarak bilinir.
- 64-bit adres alanı desteği, 2^{64} byte adreslenebilir bellek sağlar; (>16 eksabayt)
 - 64-bit teoride 2^{64} byte adreslese de, pratikte adresleme 64-bitten daha azdır.
 - X86-64 mimarisi, dört seviyeli paging hiyerarşisi kullanarak 4 KB, 2 MB veya 1 GB sayfa boyutlarını destekleyen 48-bit mantıksal adres sağlamaktadır.
 - Bu adresleme şeması PAE kullanabildiğinden, mantıksal adresler 48 bit boyutundadır ancak 52-bit fiziksel adresleri (4.096 terabayt) destekler.



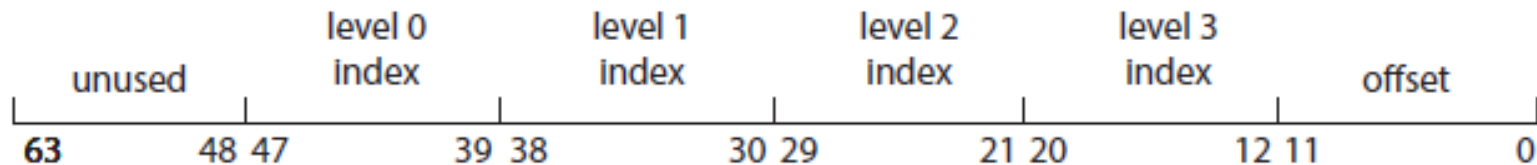
ARMv8

- Intel yongaları PC pazarına 30 yıldan fazladır hakimdir.
 - Ancak akıllı telefon ve tablet gibi mobil cihazlar için yongalar genellikle ARM'dır.
- Intel hem çip tasarlar hem de üretirken, ARM yalnızca tasarlar.
 - ARM mimari tasarımları, çip üreticilerine lisanslanır.
 - Apple, iPhone ve iPad cihazları için ARM tasarımını lisanslamıştır.
 - Çoğu Android tabanlı akıllı telefonda ARM işlemcilerini kullanmaktadır.
 - ARM, mobil cihazlara ek olarak, gerçek zamanlı gömülü sistemlere de tasarım sağlar.
 - ARMv8'in üç çeviri granülü (**translation granules**) vardır: 4KB, 16KB ve 64KB
 - Her çeviri granülü, bölgeler (**regions**) olarak bilinen bitişik belleğin (contiguous bit) daha büyük bölümlerinin yanı sıra farklı sayfa boyutları sağlar.

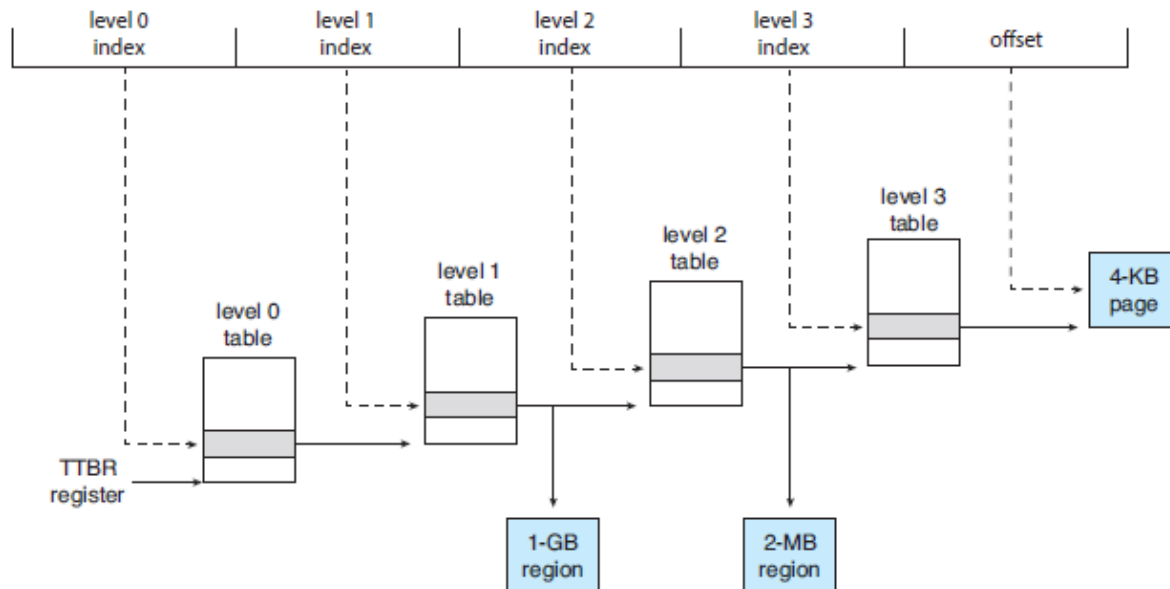
Translation Granule Size	Page Size	Region Size
4 KB	4 KB	2 MB, 1 GB
16 KB	16 KB	32 MB
64 KB	64 KB	512 MB

ARMv8

- 4-KB ve 16-KB granüller için dört seviyeye kadar paging, 64-KB granüller için üç seviyeye kadar paging kullanılabilir.
- Aşağıda 4-KB çeviri granüle sahip ARMv8 adres yapısını göstermektedir.



- TTBR register'ı (**translation table base register**), mevcut iş parçasığı için Level-0 tablosunu işaretler.



ARMv8

- Dört seviye’li bir hiyerarşik paging’in tamamı kullanılırsa, 4 KB’lik (offset 0-11 arası bitler) bir page sunulur.
- Ancak, level-1 ve level-2 tabloları, 1 GB’lık (level-1 tablosu) veya 2 MB’lık bir bölgeyi (level-2 tablosu) refere edebilirler.
 - Örneğin, level-1 tablosu level-2 tablosu yerine 1 GB’lık bir bölgeyi refere ederse, düşük değerlikli 30 bit (0-29 arası bitler) 1 GB’lık bölge içindeki offset’i ifade eder.
 - Benzer şekilde, level-2 tablosu level-3 tablosu yerine 2-MB’lık bir bölgeyi refere ederse, düşük değerlikli 21 bit (0-20 arası bitler) 2-MB bölge içindeki offset’i ifade eder.
- ARM mimarisi ayrıca iki seviyeli TLB’yi destekler. İç seviyede iki mikro TLB vardır (veriler ve komutlar için).
 - Mikro TLB, ASID’leri destekler. Dış seviyede tek bir TLB bulunur (Ana TLB).
 - Adres çevrimi mikro TLB’de başlar. Adres mikro TLB’de bulunamadığında, ana TLB kontrol edilir.
 - Her iki TLB de bulunamazsa, donanım bir page-table süreci başlatır ve adres çevrimi page-table (ana bellek) üzerinden işletilir.

Bölüm Özeti

- Bellek, modern bir bilgisayar sisteminin işleyişinin merkezidir ve her biri kendi adresine sahip geniş bir bayt dizisinden oluşur.
- Her bir sürece bir adres alanı tahsis etmenin yolu, base ve limit register'larının kullanılmasıdır. Base register, en küçük fiziksel bellek adresini tutar ve limit register'ı geçerli boyutu belirler.
- CPU tarafından üretilen adres, bellek yönetim biriminin (MMU) bellekteki fiziksel bir adrese çevirdiği mantıksal adres olarak bilinir.
- Bellek ayırmaya yönelik bir yaklaşım, çeşitli boyutlarda bitişik belleğin bölümlerini ayırmaktır. Bu bölümler, üç olası stratejiye göre tahsis edilebilir: (1) first fit, (2) best fit ve (3) worst fit.
- Modern işletim sistemleri, belleği yönetmek için paging kullanır. Paging'de fiziksel bellek: frame adı verilen sabit boyutlu bloklara, mantıksal bellek: page olarak adlandırılan aynı boyuttaki bloklara bölünür.
- Paging kullanıldığında, mantıksal bir adres iki bölüme ayrılır: page-number ve page-offset. page-number, sayfayı tutan fiziksel bellekteki çerçeveyi, Page-offset ise referans edilen çerçevedeki konumu tanımlar.

Bölüm Özeti

- TLB, page-table için donanım önbelleğidir. Her TLB kaydı bir page-number ve ona karşılık gelen frame numarasını içerir.
- Paging sistemlerinde adres çevriminde page-number'ın TLB'de olup olmadığının kontrol etmesi gerektirir. TLB'de ise frame number TLB'den alınır. Yoksa page-table'dan alınır.
- Hiyerarşik sayfalama, bir mantıksal adresin her biri farklı düzeylerdeki page-table'ları referans ettiği birden çok parçaya bölünmesini ifade eder. Adresler 32 bitin üzerine çıktıkça, hiyerarşik düzeylerin sayısı artabilir. Bu sorunu ele alan iki teknik, Hashing page-table ve Inverted page table'dir.
- Swapping, sistemin multiprogramming düzeyini artırmak için süreçlere ait sayfaların diske taşınmasını ifade eder.
- Intel 32-bit mimarisinde iki seviyeli page-table kullanılır. 4-KB veya 4-MB page-size'ı destekler. Bu mimari aynı zamanda 32 bit işlemcilerin 4 GB'den büyük fiziksel adres uzayına erişmesine izin veren PAE'yi (page address extension) destekler. X86-64 ve ARMv8 mimarileri, hiyerarşik sayfalama kullanan 64-bit mimarilerdir.

Çalışma Soruları

1. 1KB'lık bir sayfa boyutu (page-size) olan bir sistemde, 3085 (decimal) adresinin sayfa numarası (page number) ve offseti (page offset) ne olur?
2. SCU-OS işletim sistemi 21 bitlik bir mantıksal adrese ve 2KB page-size'a sahiptir. Ancak üzerinde koştugu makine 16 bitlik bir fiziksel adrese sahiptir. Bu tanımlamalara göre sistemde aşağıdaki page-table yapıları kullanıldığında kaç page-table girişi olur?
 - a. Geleneksel, tek seviyeli bir page-table için
 - b. Inverted page-table için
 - c. SCU-OS işletim sistemindeki maksimum fiziksel bellek miktarı nedir?
3. 4KB page-size'a sahip, 64 frame fiziksel belleğe eşlenmiş 256 sayfalık mantıksal adres alanı için;
 - a. Mantıksal adres için kaç bit gereklidir?
 - b. Fiziksel adres için kaç bit gereklidir?
4. iOS ve Android gibi mobil işletim sistemlerinde takaslamanın (swapping) neden desteklenmediğini açıklayın.